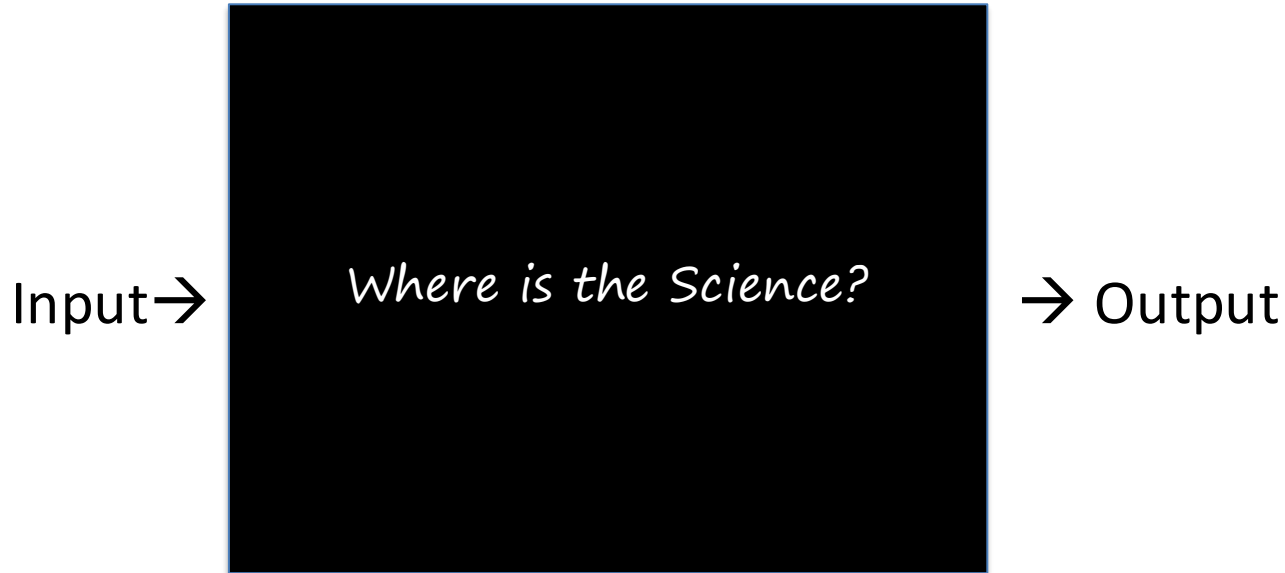


# *The trials and errors of physically informing neurons*



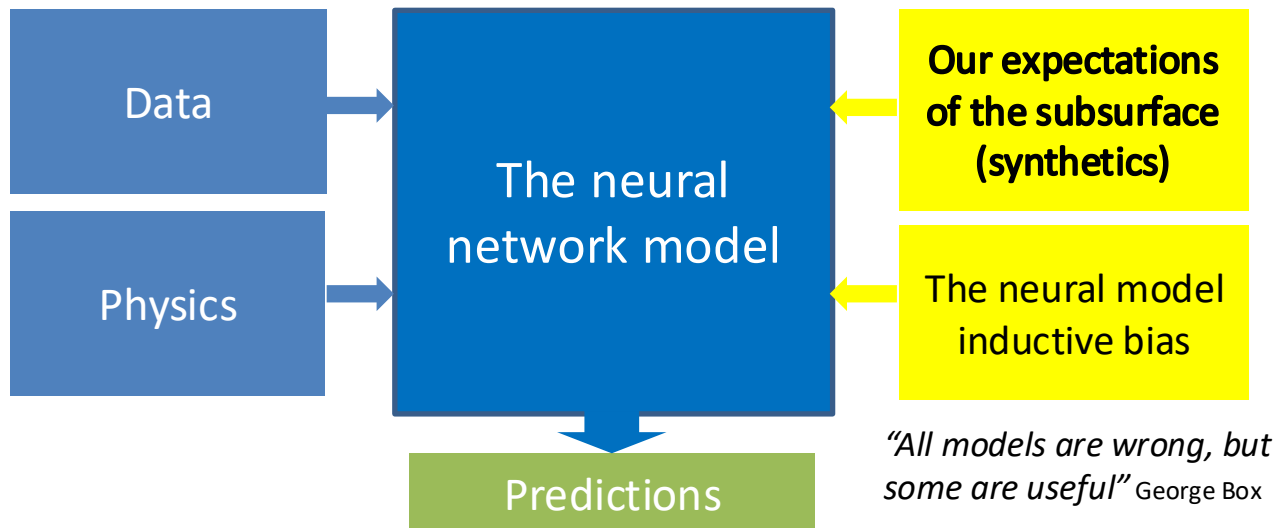
Many scientific applications of ML has been agnostic to the underlying physics → Black box



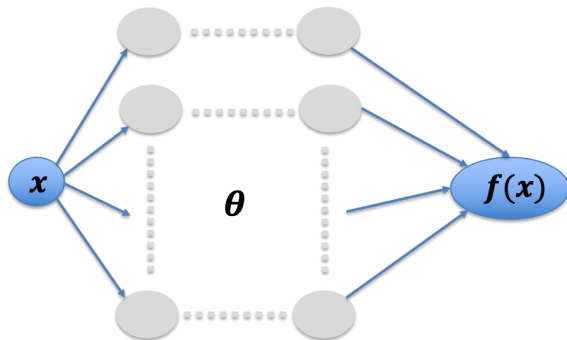
Just doing interpolation?  
Maybe extrapolation

# Our goal

An optimization framework



# What is neural representation? An NN function



Continuous and meshless

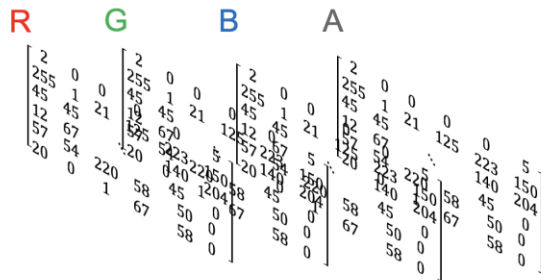
$$Loss = D(f_{\theta}(x), f(x_i))$$

Automatic differentiation (AD):

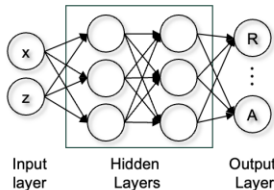
$$\frac{\partial f}{\partial x} = \nabla_{\theta} f(x; \theta) \cdot \frac{\partial \theta}{\partial x}$$



Grid  
Representation



Implicit  
Neural  
Representation



Sitzmann, V., Martel, J. N. P., Bergman, A. W., Lindell, D. B., & Wetzstein, G. (2020). Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)* (pp. 7462–7473). Curran Associates, Inc.

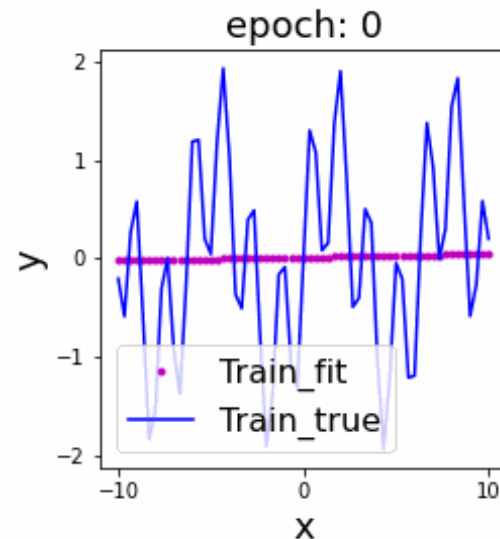
# Universal approximation theorem

The approximation capabilities of the feedforward architecture on the space of continuous functions between two Euclidean spaces

$f_\epsilon : \mathbb{R}^d \rightarrow \mathbb{R}^D$  with representation

$$f_\epsilon = W_2 \circ \sigma \bullet W_1,$$

$$\sup_{x \in K} \|f(x) - f_\epsilon(x)\| < \epsilon$$



Pinkus (1999)

# Physics informed neural networks (PINNs)

*incorporate physical laws, represented by partial differential equations (PDEs), into the loss function of a neural network. This allows for solving forward and inverse problems in physics.*

## Why Use PINNs?

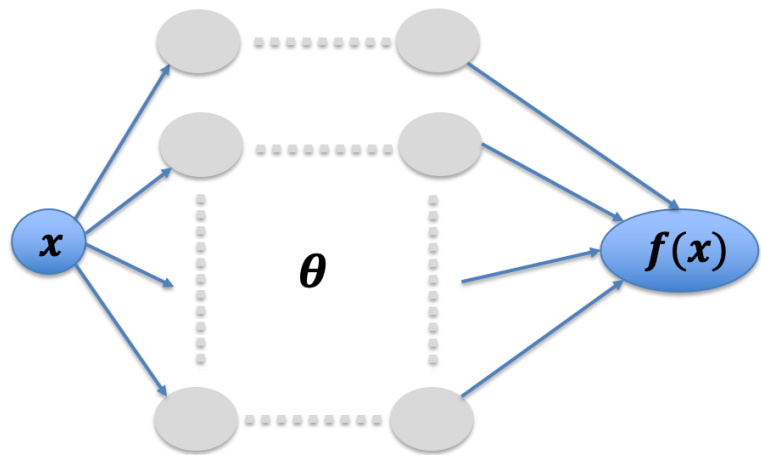
- Enforce physics constraints without large labeled datasets -- unsupervised
- Solve PDEs efficiently?
- Useful in fluid dynamics, material science, electromagnetics, seismic waves etc.

# *Partial differential equation (PDE)*

is a type of mathematical equation that involves partial derivatives of an unknown function with respect to multiple variables. PDEs are used to describe various physical phenomena, such as heat conduction, wave propagation, fluid dynamics, and quantum mechanics.



The framework (Raissi et al., 2019)



$$Loss = |PDE(f(x; \theta), a)|$$

A fully connected MLP:

$$u_{\theta}(\mathbf{x}, t) = W_0 \sigma_L(W_L \sigma_{L-1}(\dots \sigma_1(W_1 \mathbf{x} + b_1)\dots) + b_L)$$

- Nonlinear activation functions improve expressiveness.
- Last layer we often use linear activation functions

# Mathematical Formulation

- Consider a physical problem, and the corresponding PDE
- $PDE(u(\mathbf{x},t)) = 0$ , where  $u(\mathbf{x},t)$  is the solution
- PINNs approximate  $u(\mathbf{x},t)$  using an NN,  $u_{\theta}(\mathbf{x},t)$ . The residual of the PDE is minimized in the loss function.

# The loss function

$$MSE = MSE_u + MSE_f$$

$$MSE_u = \frac{1}{N_u} \sum_{i=1}^{N_u} |u(t_u^i, x_u^i) - u^i|^2$$

$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2$$

Raissi et.al. (2019)

# *Automatic Differentiation (AD)*

- AD efficiently (?) computes derivatives using the chain rule:

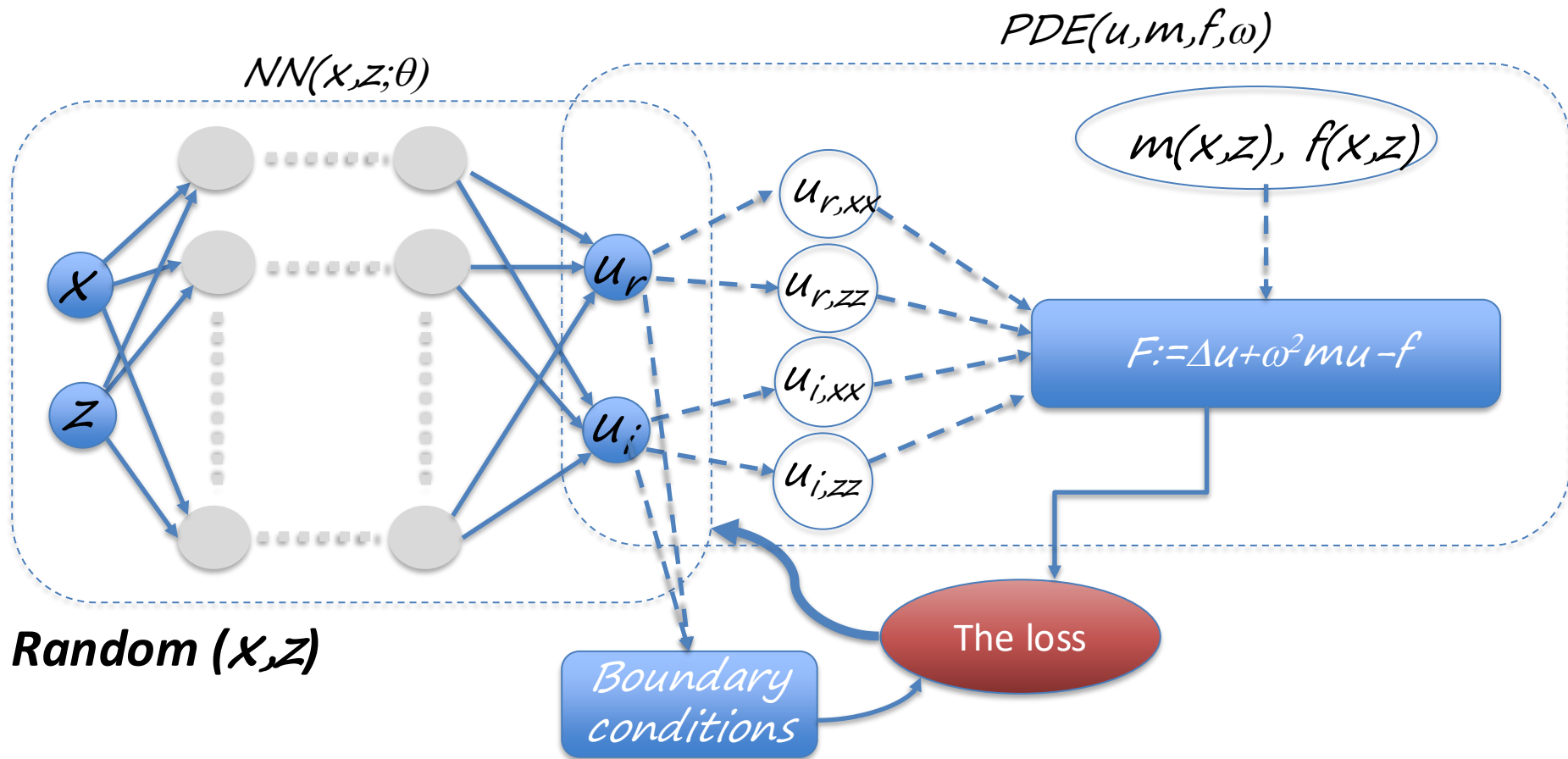
$$df/dx = \sum_i \partial f / \partial h_i \partial h_i / \partial x$$

- Enables enforcement of PDE constraints.

# Training Process

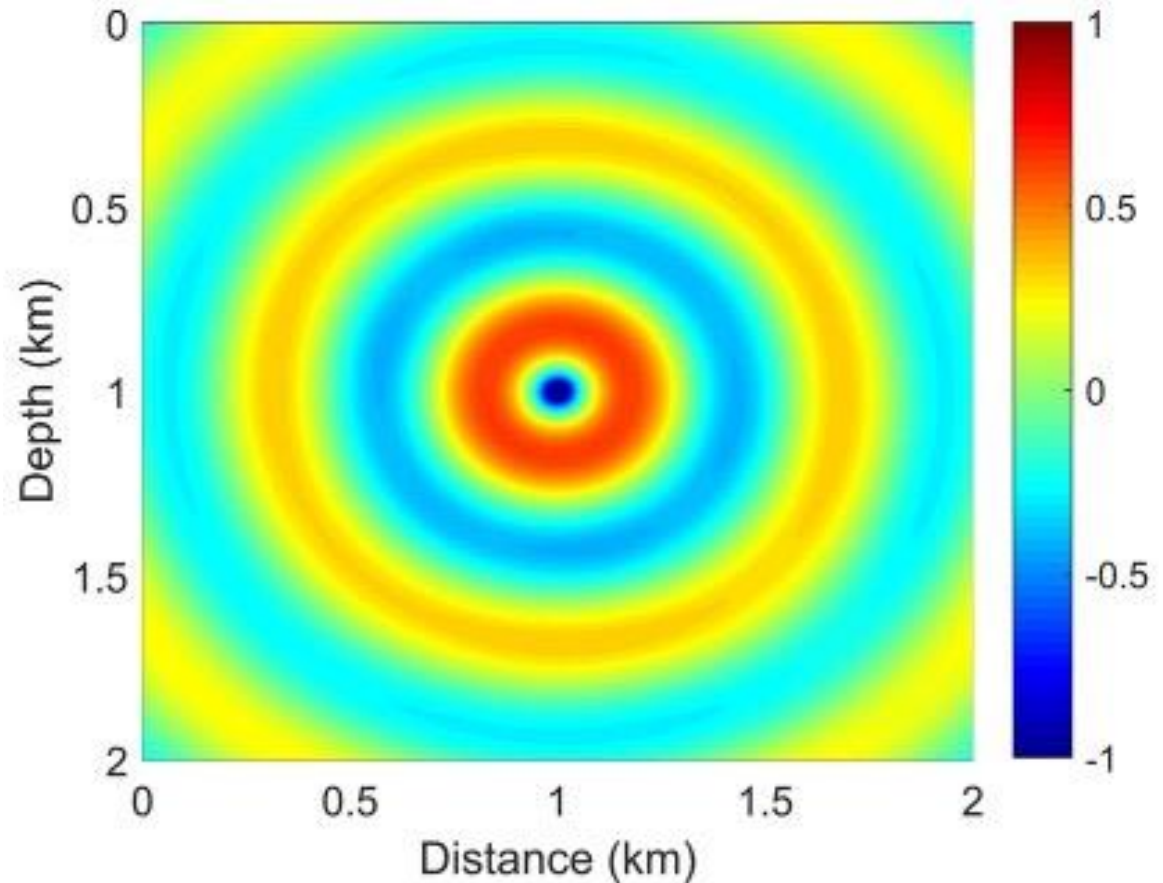
- Define neural network  $u_{\theta}(\mathbf{x}, t)$ , usually an MLP
- Pick points from the solution domain to train the network
- Measure the loss through Automatic Differentiation (AD)
- Minimize loss using gradient-based optimization
- Evaluate accuracy with analytical/numerical solutions

# The network (Helmholtz equation)

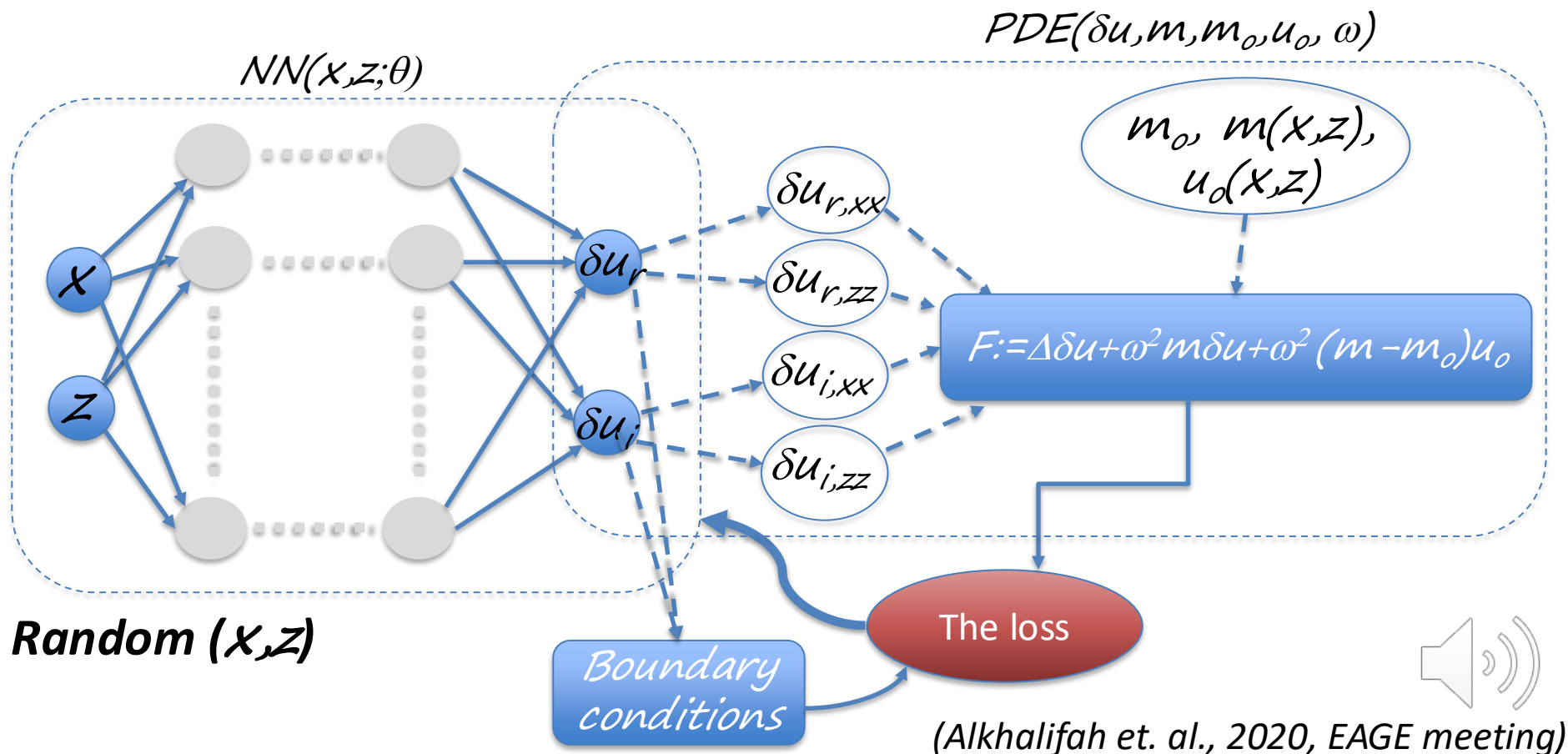


# *The source singularity*

*Requires more  
samples near the  
source, where the  
gradient is not  
defined*



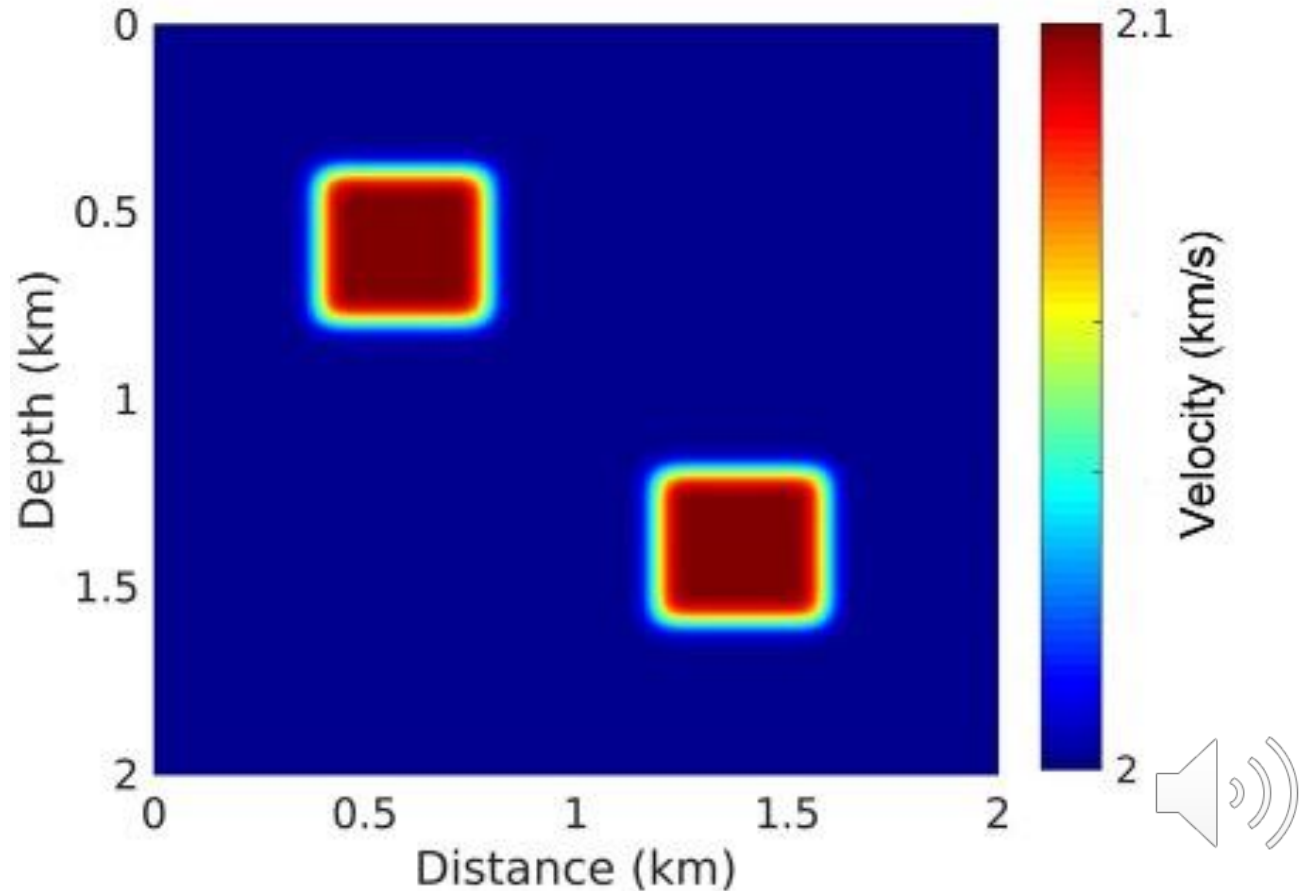
# The scattered wavefield





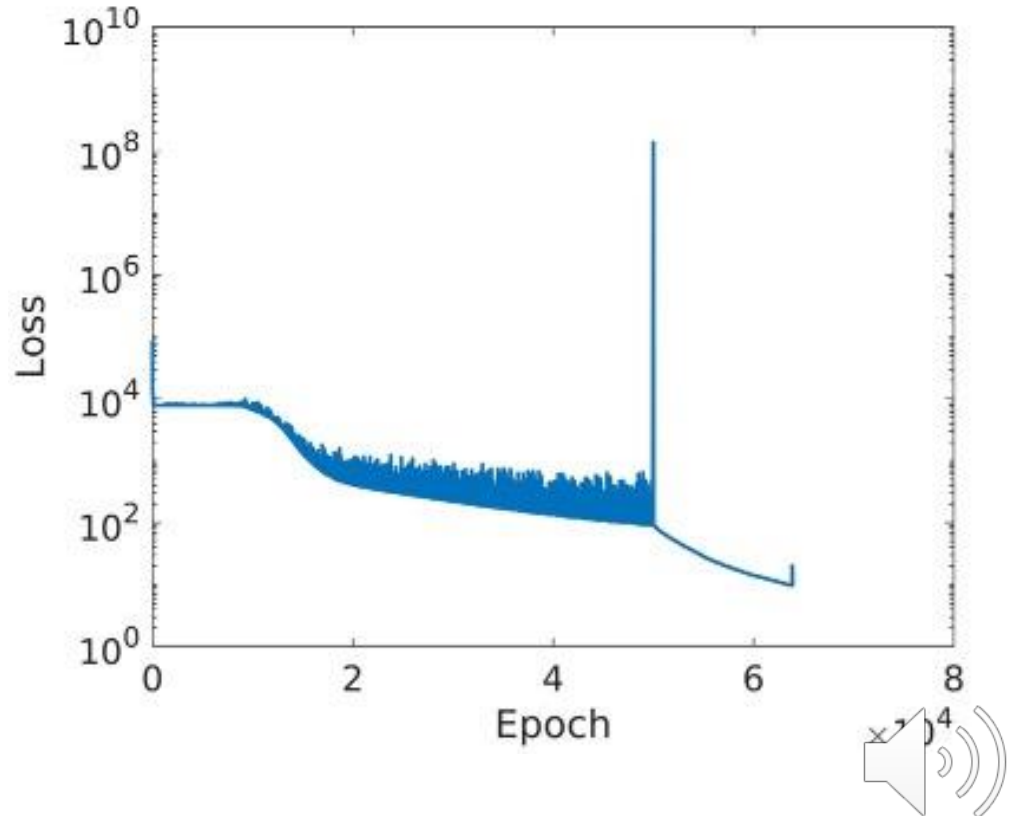
# *Two box model, homogenous background*

*The background velocity is 2 km/s*

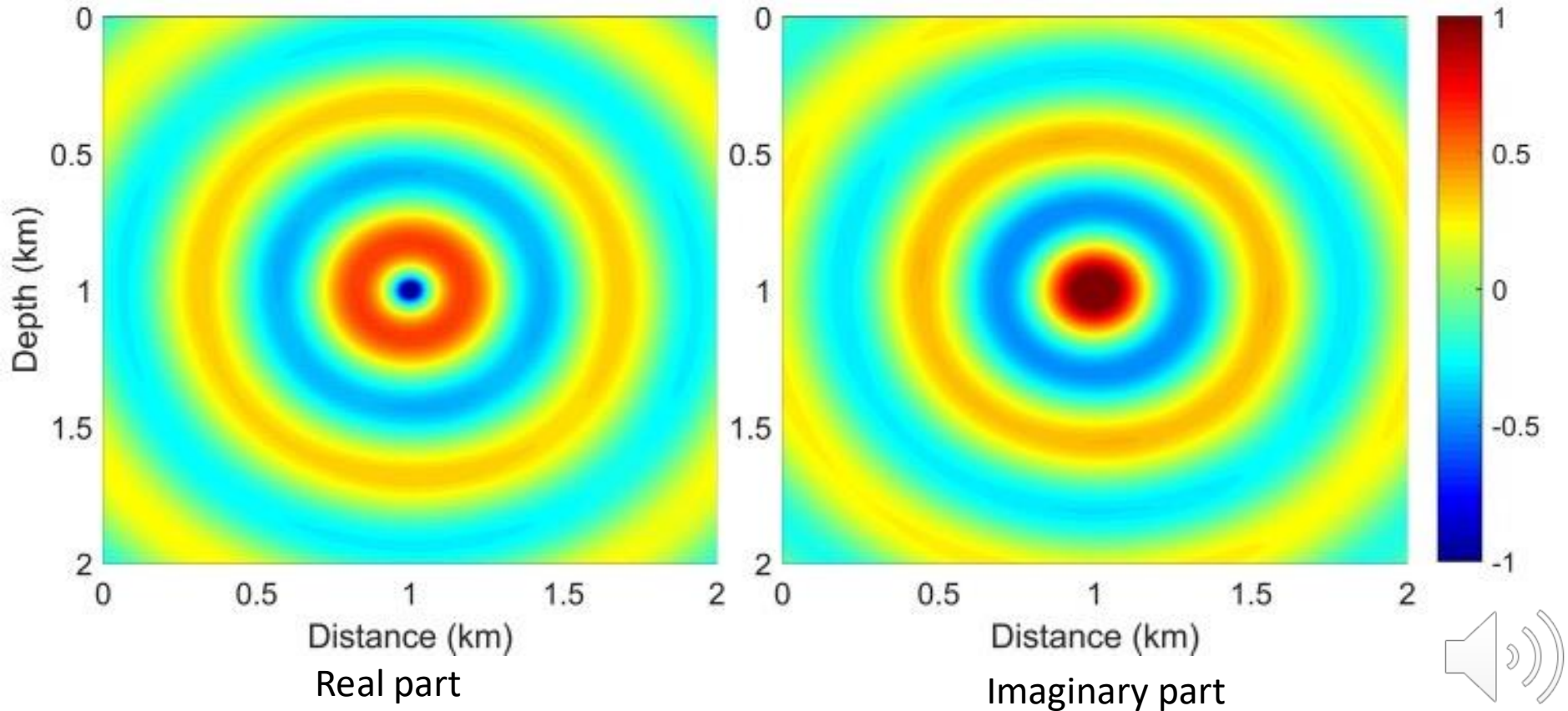


# *The training (4Hz)*

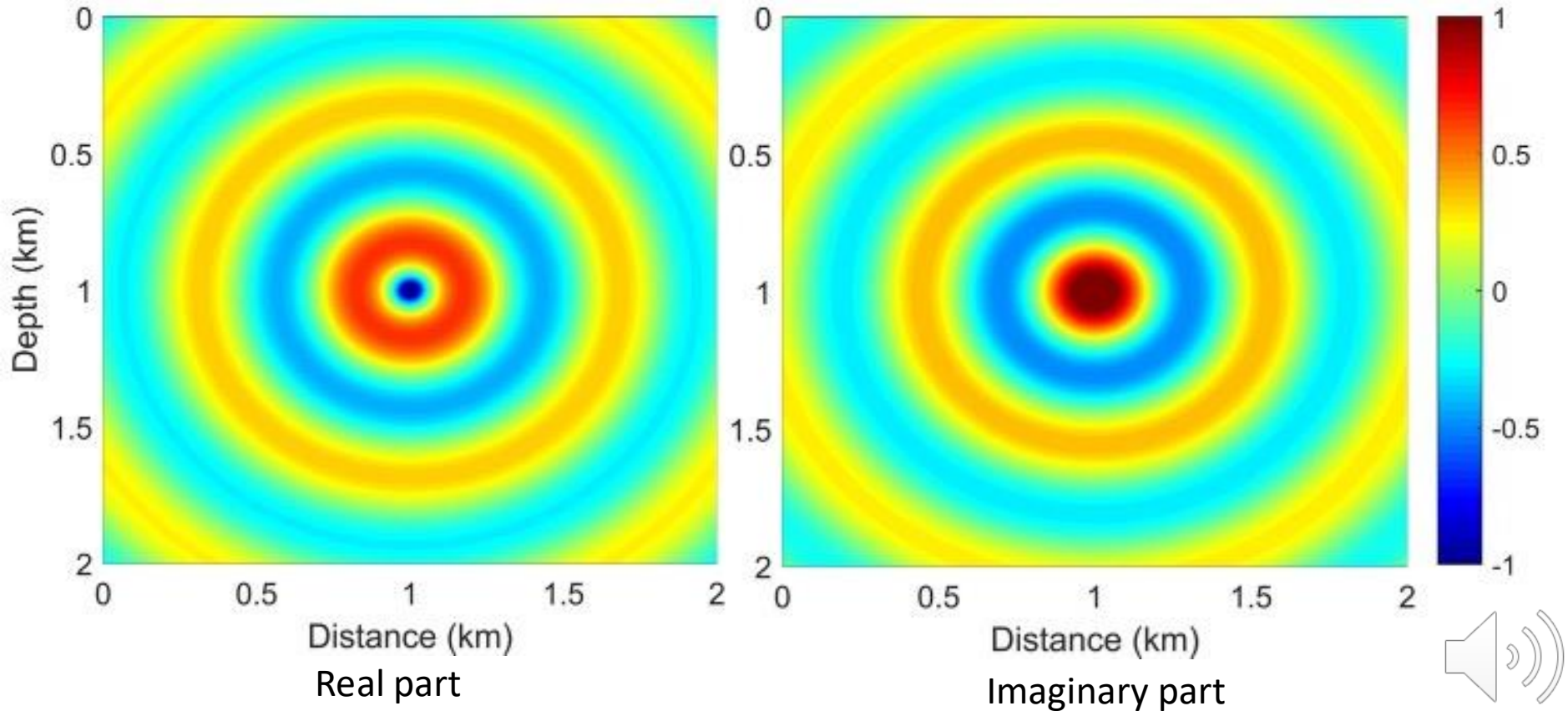
- *8-hidden-layers, 20 neurons*
- *5000 random samples*
- *Full batch training*
- *50000 Adam iterations*
- *20000 l-BFGS iterations*
- *Inverse tangent activation function here and throughout*



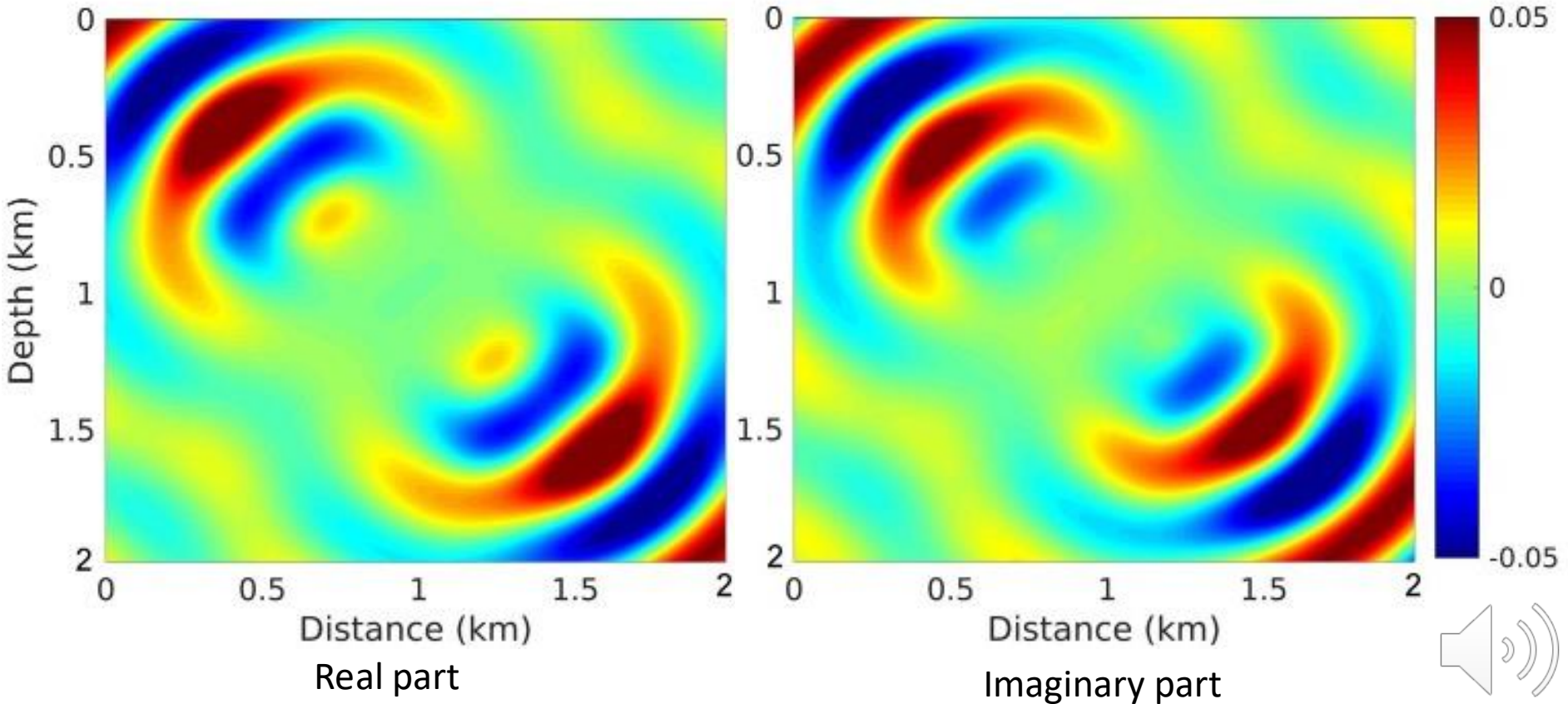
# *The numerical 4 Hz wavefield*



# *4 Hz wavefield, homogeneous background 2 km/s*

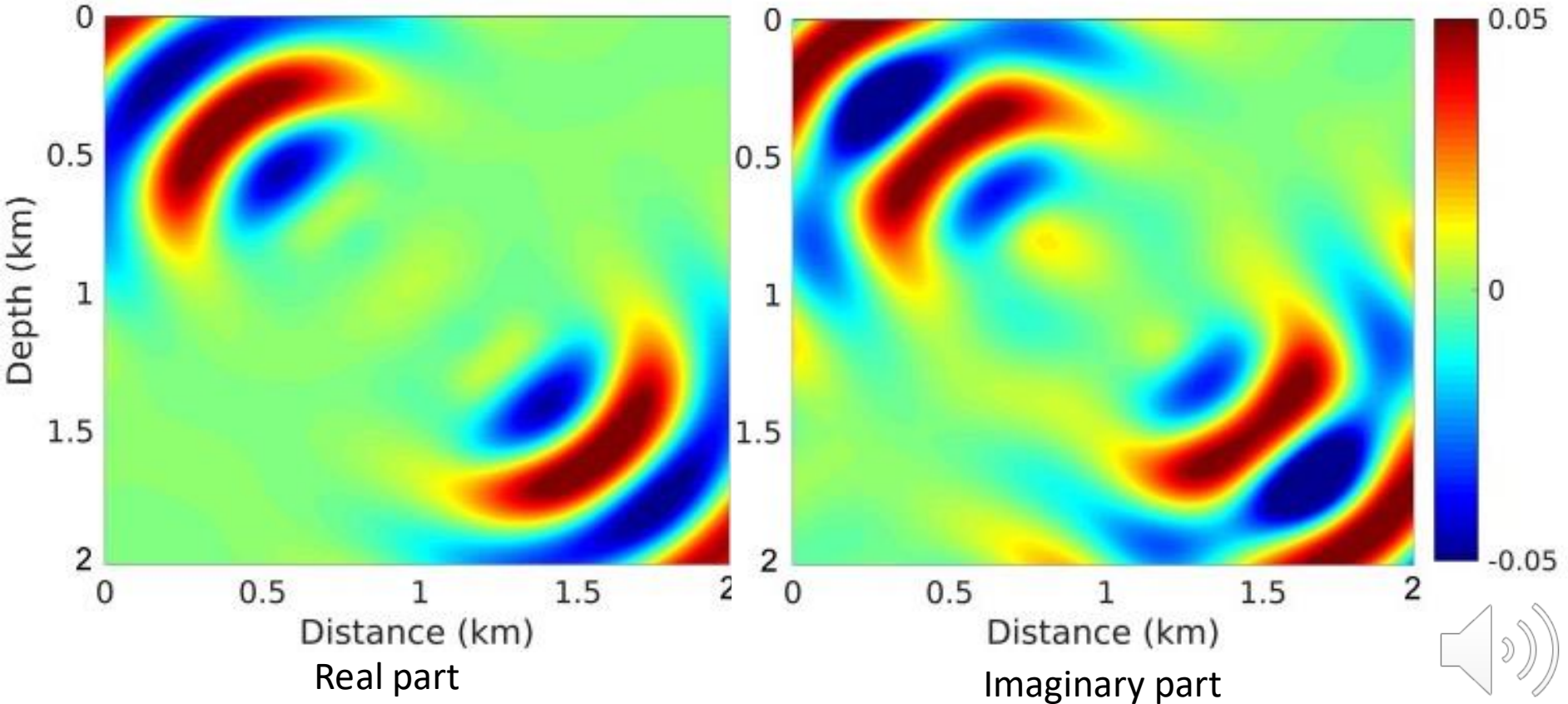


# *The numerical 4 Hz scattered wavefield*

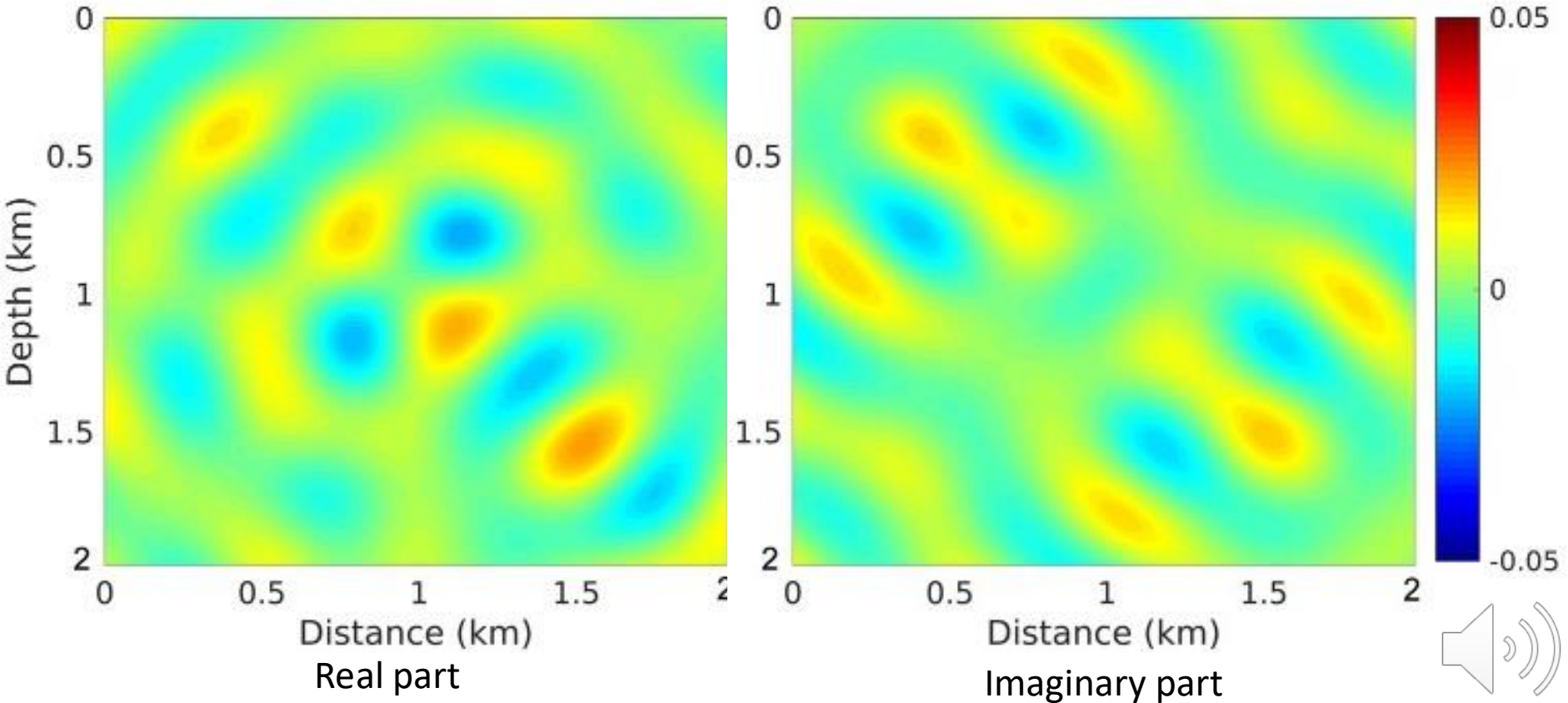




# *The NN 4 Hz scattered wavefield*

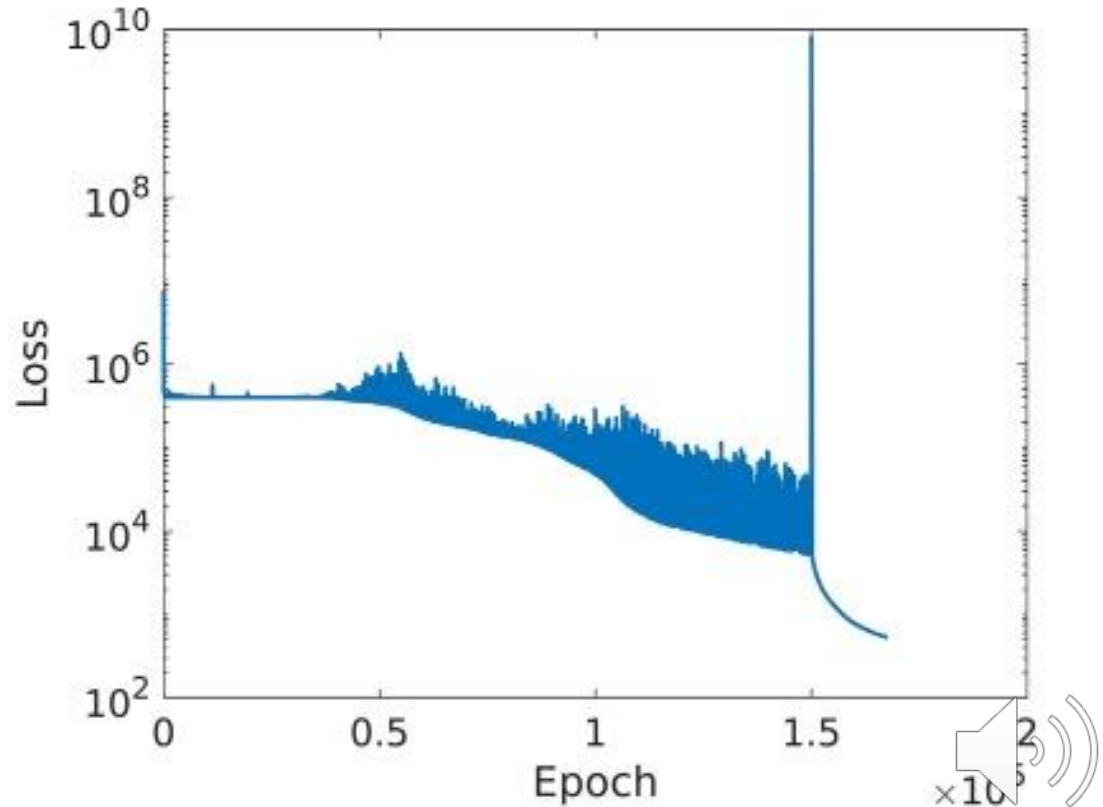


# *The difference, plotted at the same scale*



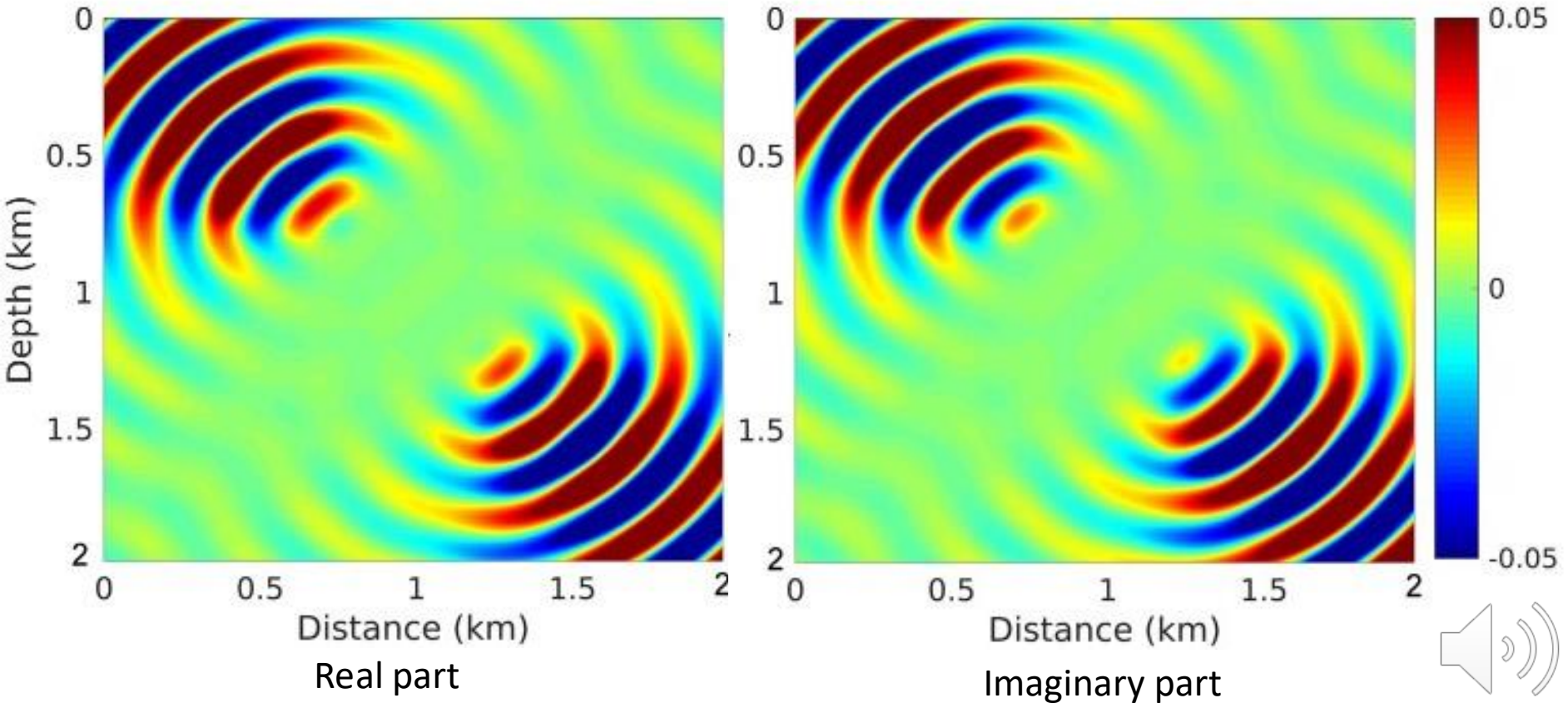
# *The training (8Hz)*

- *8-layers, 40 neurons*
- *10000 random samples*
- *Full batch training*
- *150000 Adam iterations*
- *20000 l-BFGS iterations*
- *Cost increase over the 4Hz is about 100%*

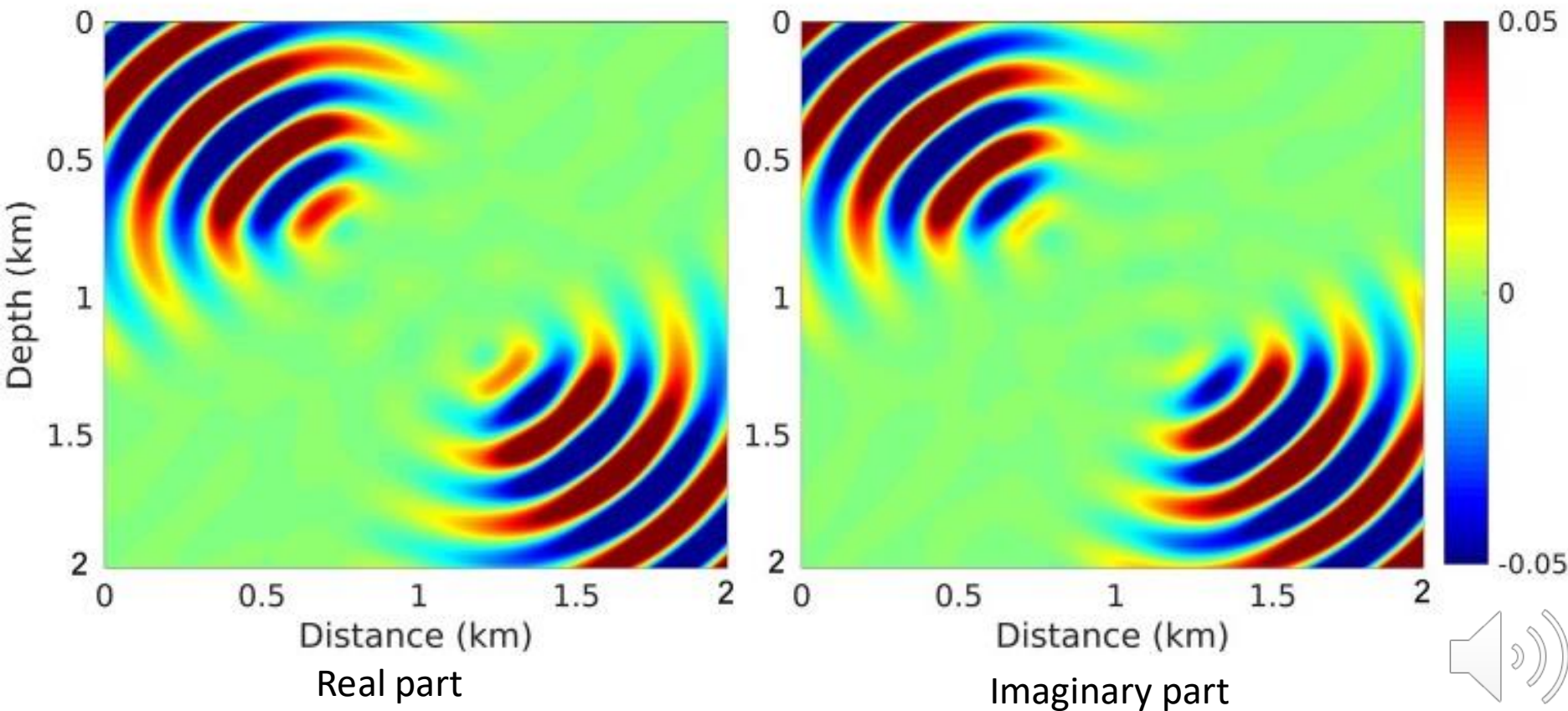




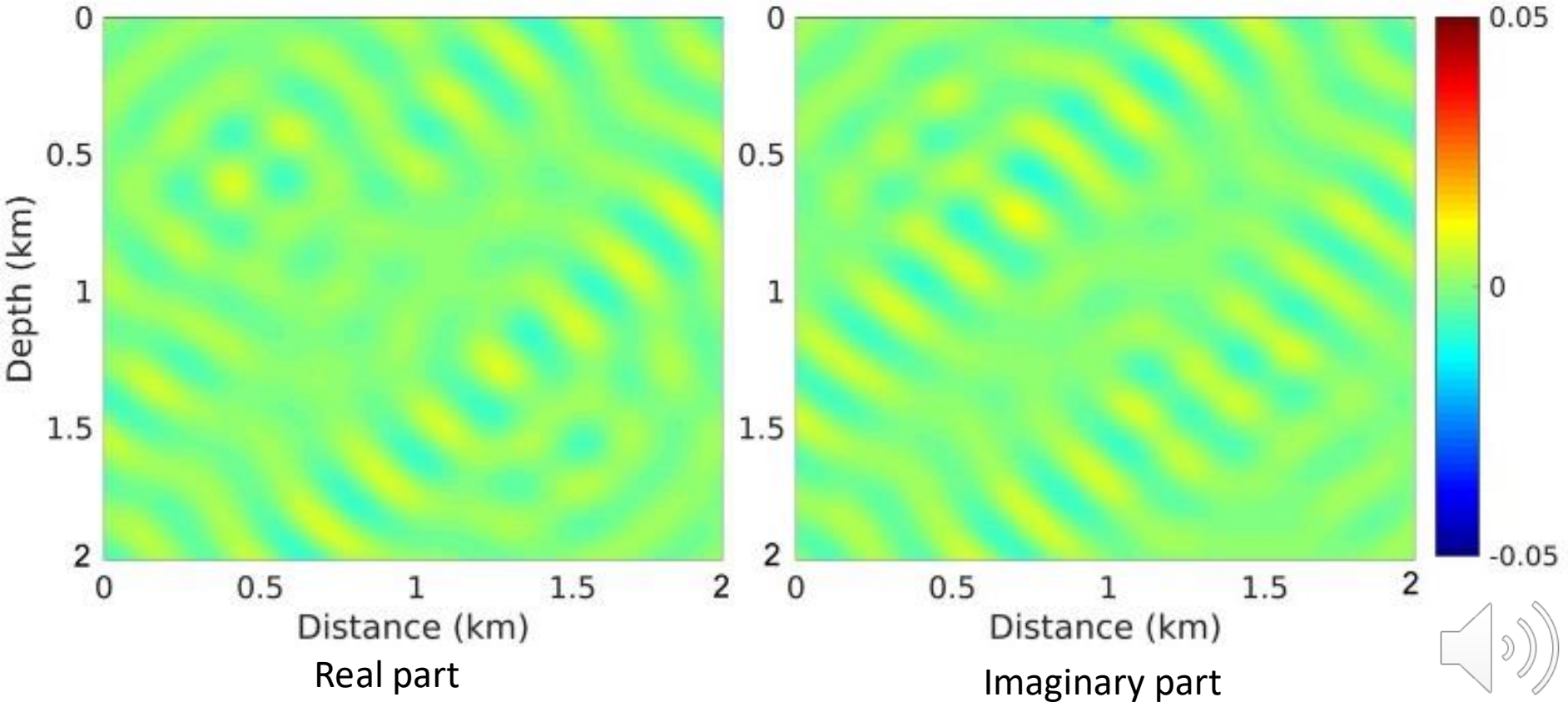
# *The numerical 8 Hz scattered wavefield*



# *The NN 8 Hz scattered wavefield*



# *The difference, plotted at the same scale*



# ***Green's functions***

- The inverse of the impedance matrix

$$\left( \nabla^2 + \frac{\omega^2}{v^2} \right) G(\mathbf{x}; \mathbf{x}_s) = \delta(\mathbf{x} - \mathbf{x}_s)$$

$$\delta G = G - G_0 \quad \delta m = \frac{1}{v^2} - \frac{1}{v_0^2}$$

$$\left( \nabla^2 + \frac{\omega^2}{v^2} \right) \delta G(\mathbf{x}; \mathbf{x}_s) = -\omega^2 \delta m G_0(\mathbf{x}; \mathbf{x}_s)$$



# ***The Loss function***

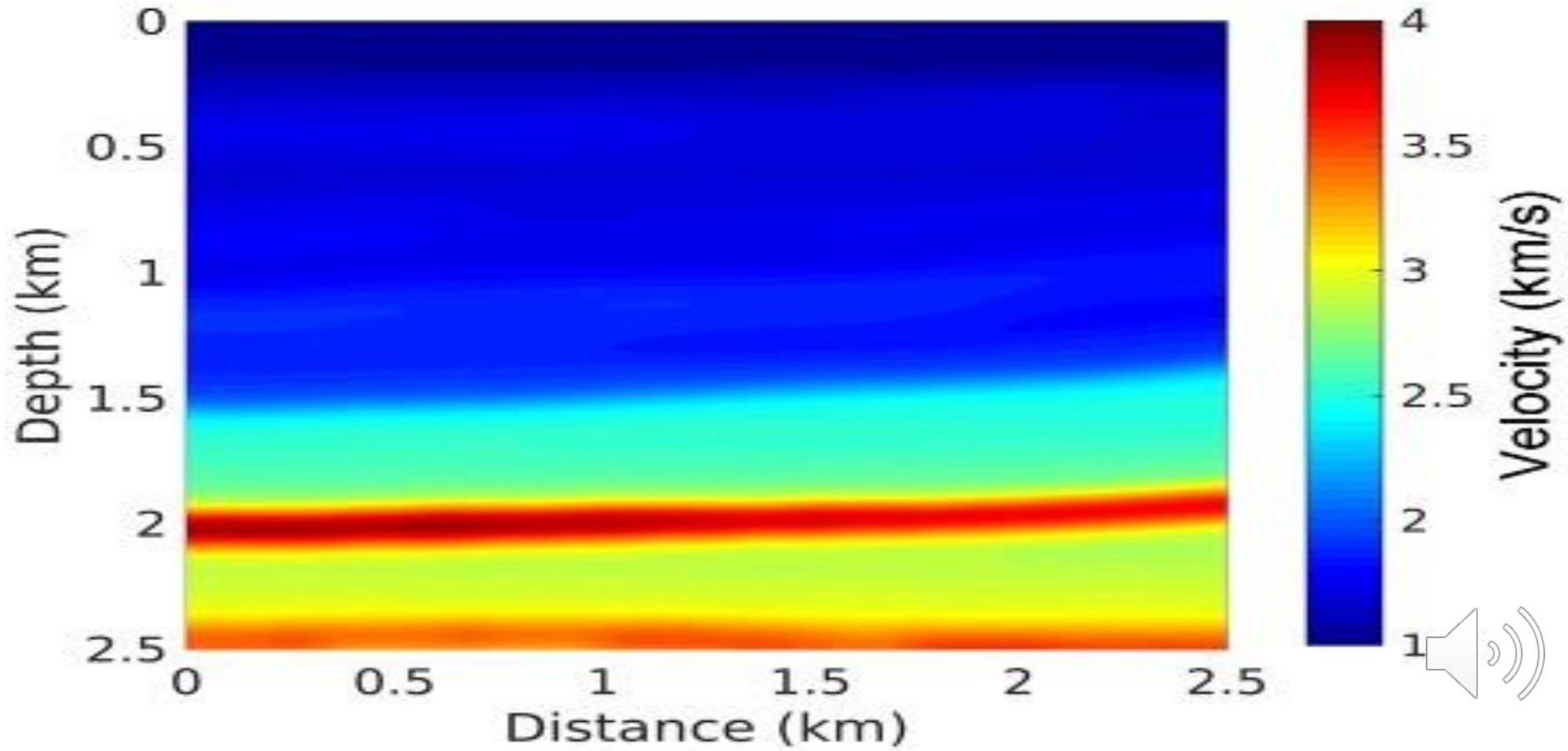
$$f = \frac{1}{N} \sum_{i=1}^N \left| \omega^2 m^{(i)} \delta G^{(i)} + \nabla^2 \delta G^{(i)} + \omega^2 \delta m^{(i)} G_0^{(i)} \right|_2^2$$

Inputs to the network:  $\mathbf{x}, \mathbf{x}_s$

In 2D, considering surface sources:  $x, z, x_s$

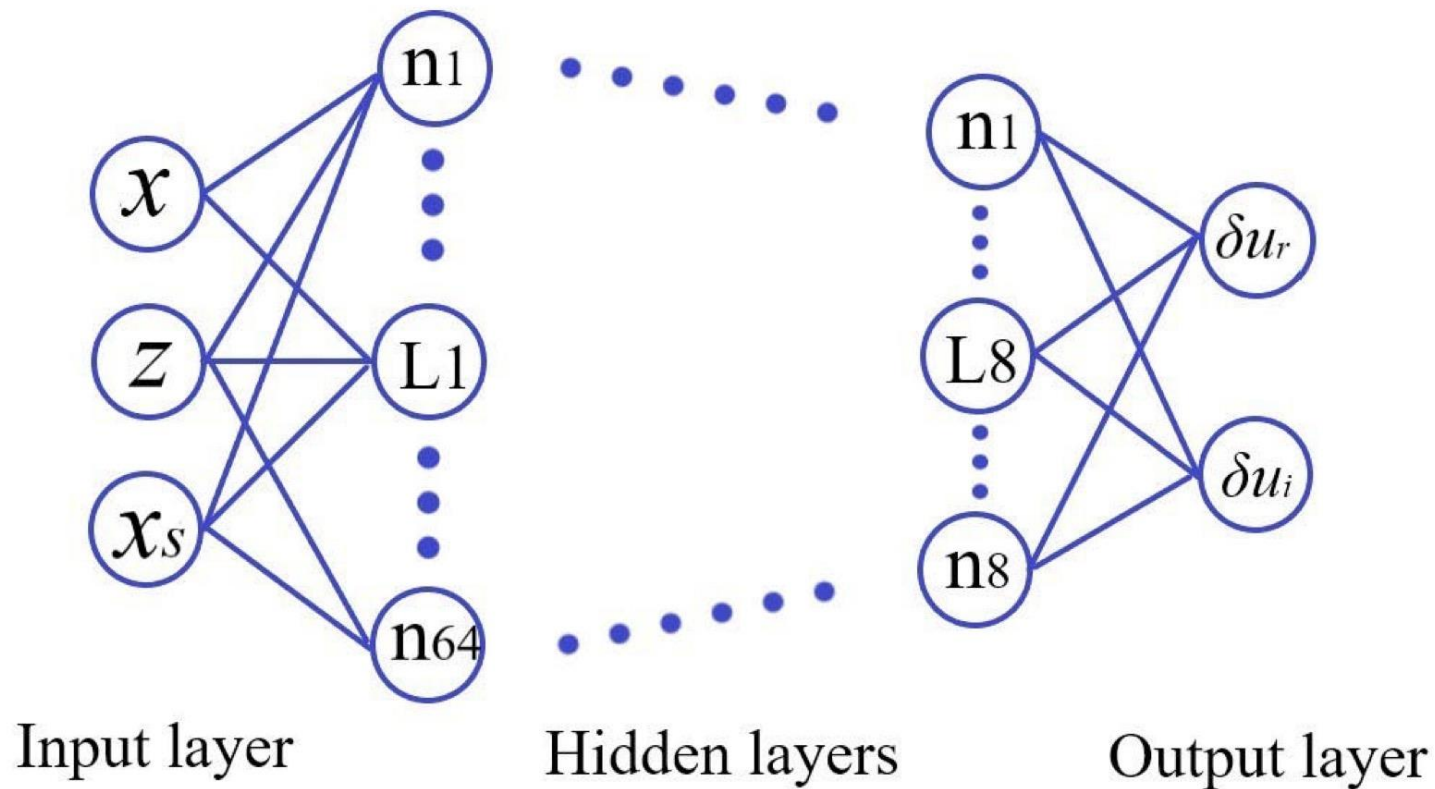


# *The velocity model*





# The network

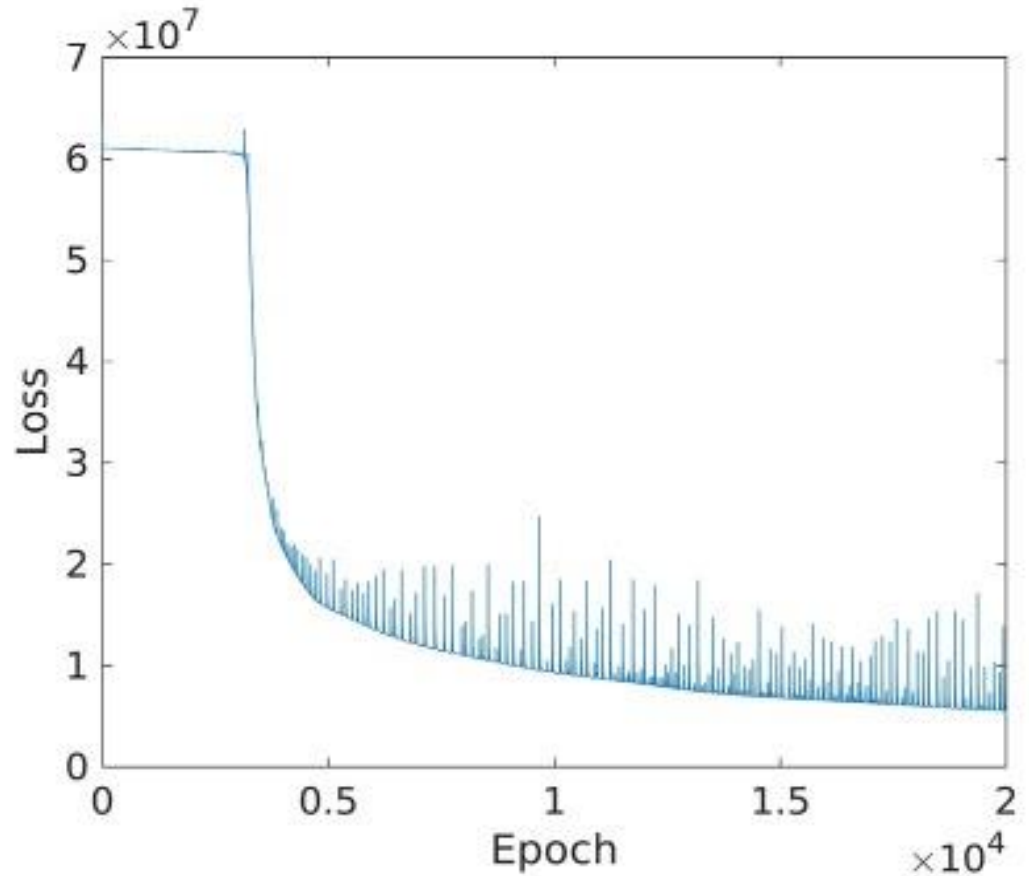


$$\delta G(x, z; x_s)$$



# The training

- **5Hz scattered wavefield**
- **8-layers**
- **$\{64,64,32,32,16,16,8,8\}$**
- **10000 random samples in  $(x,z,x_s)$**
- **Full batch training**
- **20000 Adam iterations**

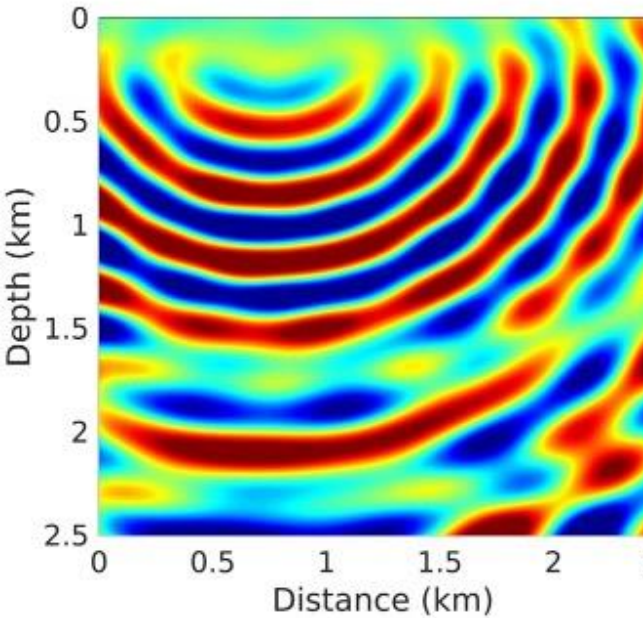


**10000 random training samples  $(x,z,x_s)$**

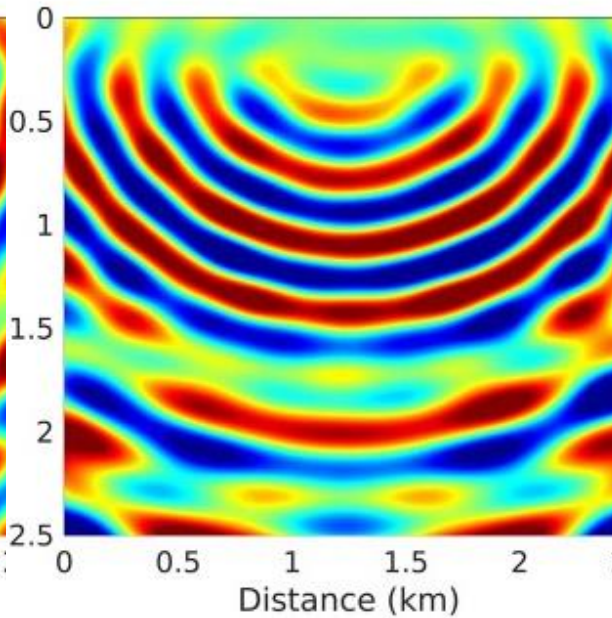




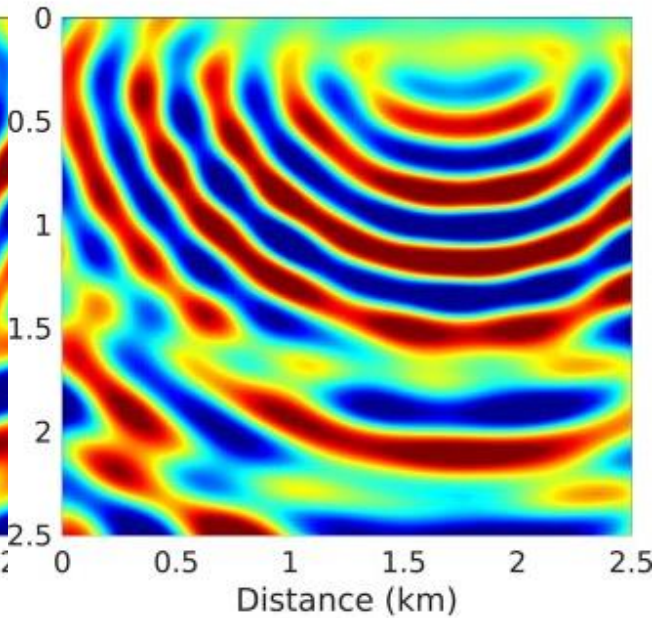
# ***The numerical scattered wavefield (real part)***



**$S_x = 0.75$  km**



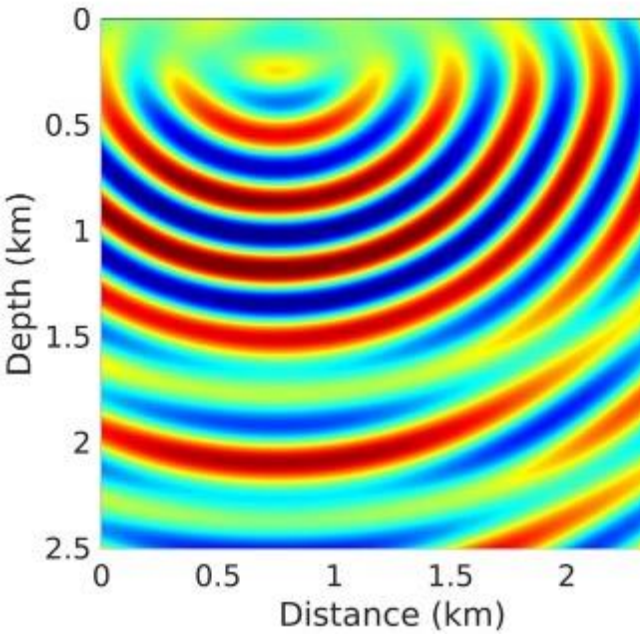
**$S_x = 1.25$  km**



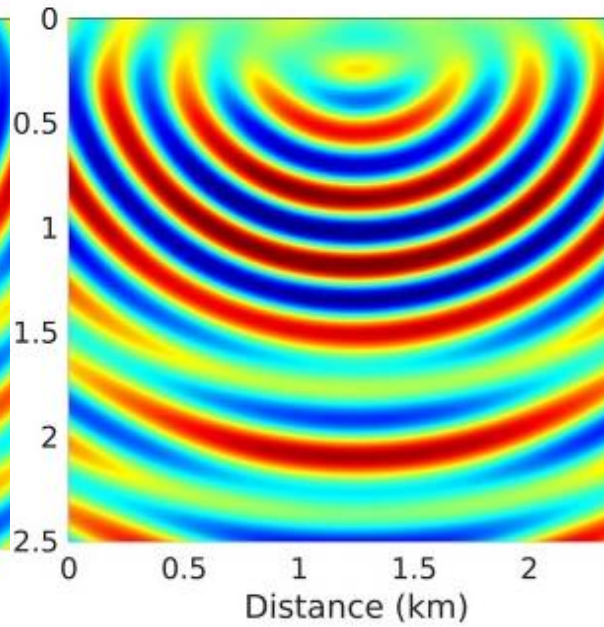
**$S_x = 1.75$  km**



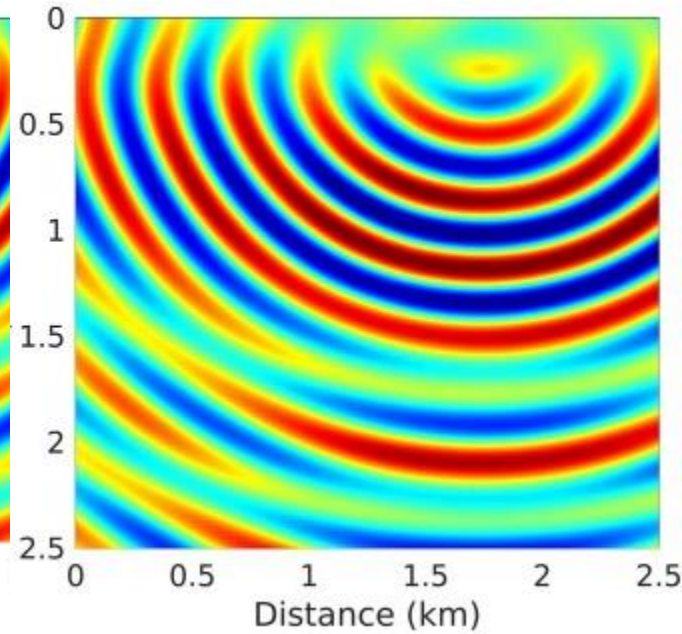
# ***The NN predicted scattered wavefield (real part)***



**$S_x = 0.75$  km**



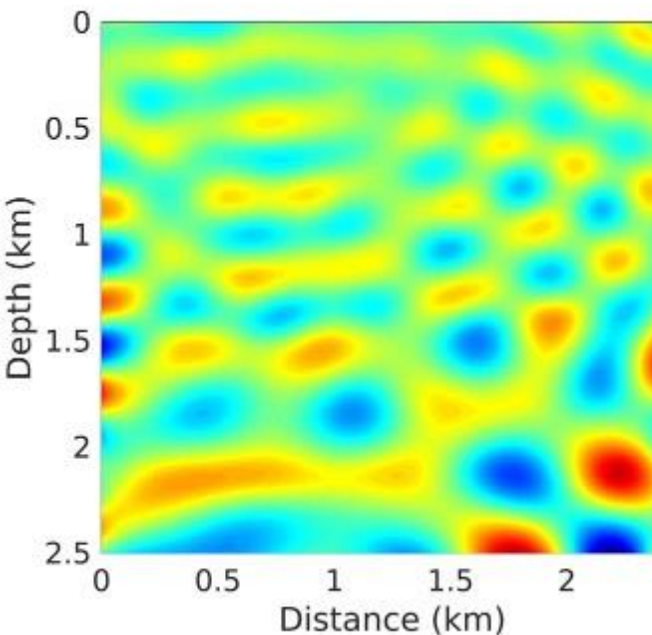
**$S_x = 1.25$  km**



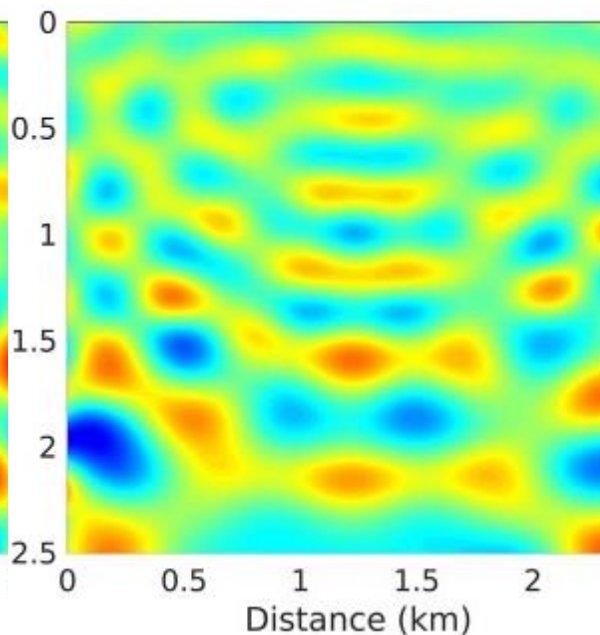
**$S_x = 1.75$  km**



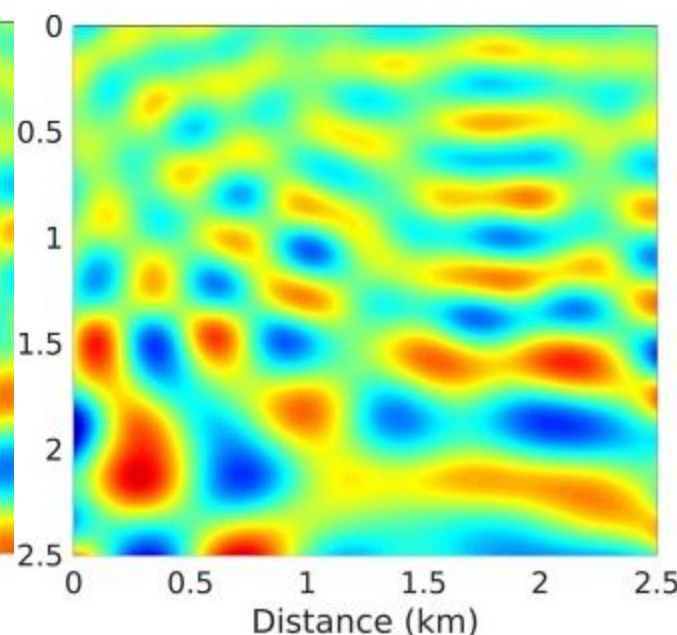
# ***The difference (real part)***



**$S_x = 0.75$  km**



**$S_x = 1.25$  km**

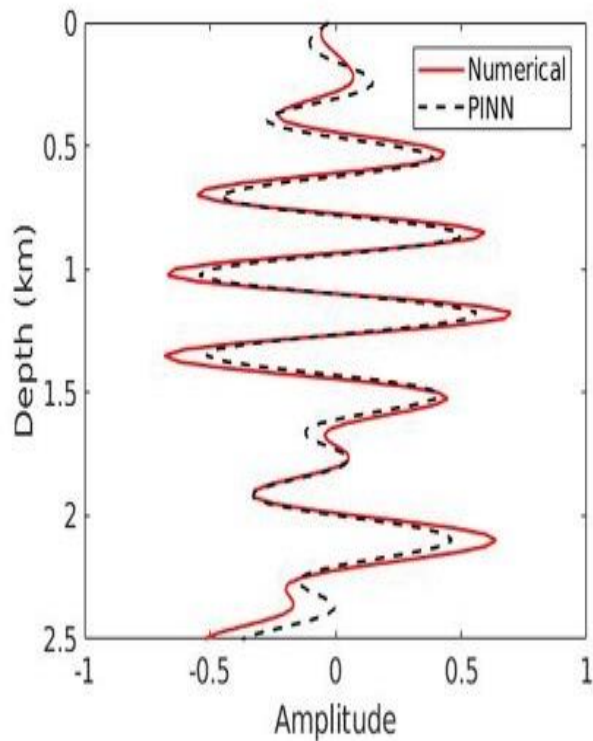


**$S_x = 1.75$  km**

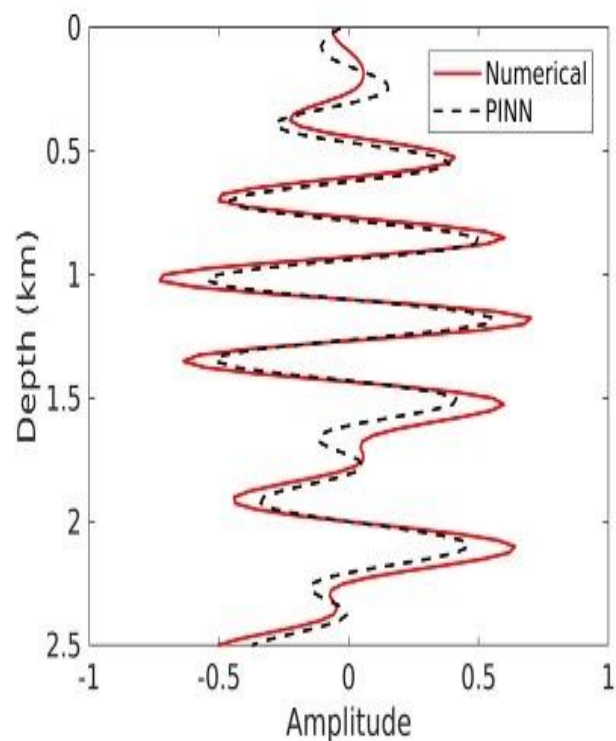




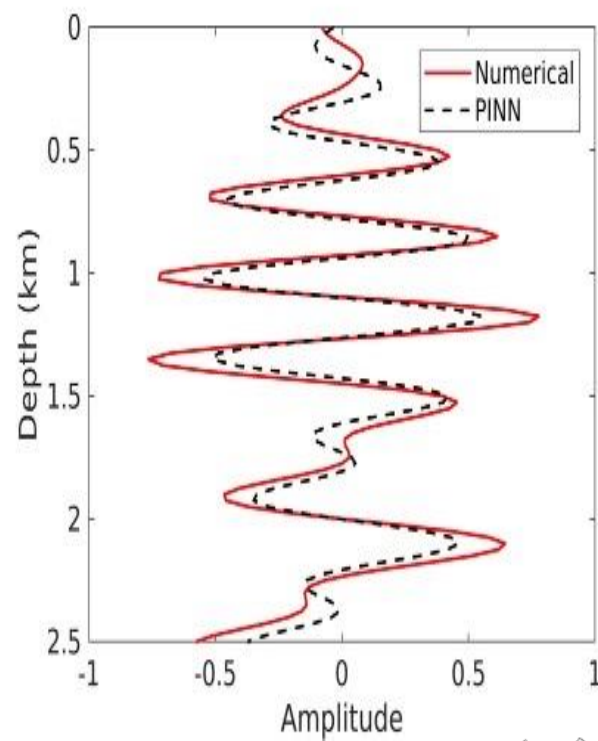
# ***The difference (real part)***



**$S_x = 0.75$  km**



**$S_x = 1.25$  km**

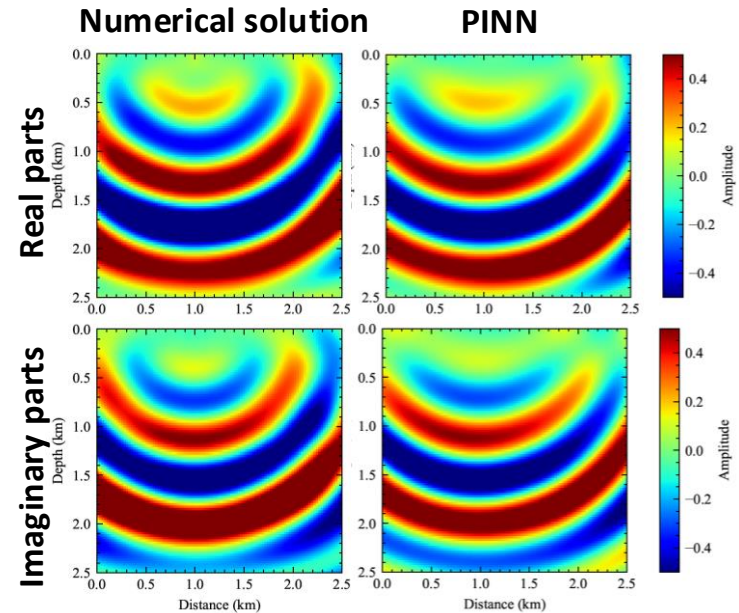
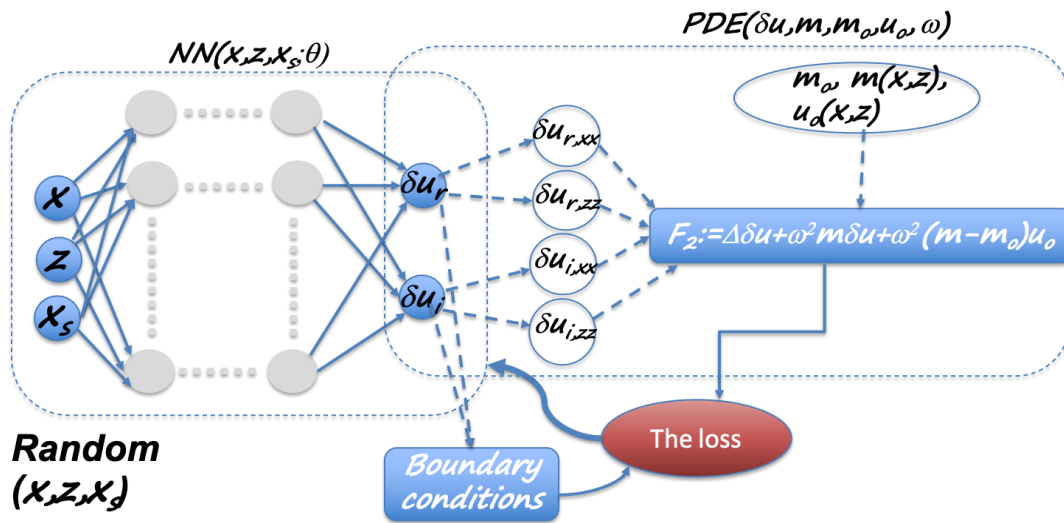


**$S_x = 1.75$  km**



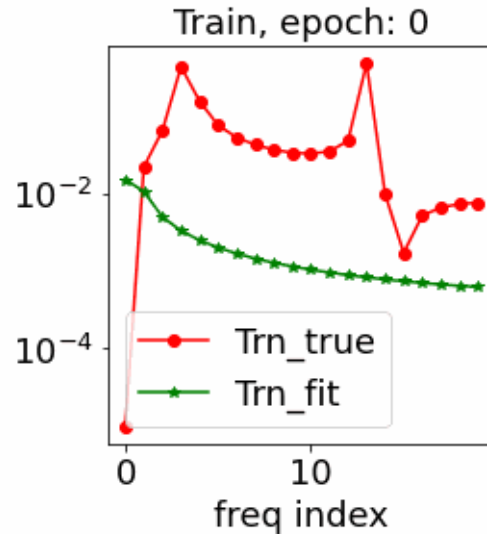
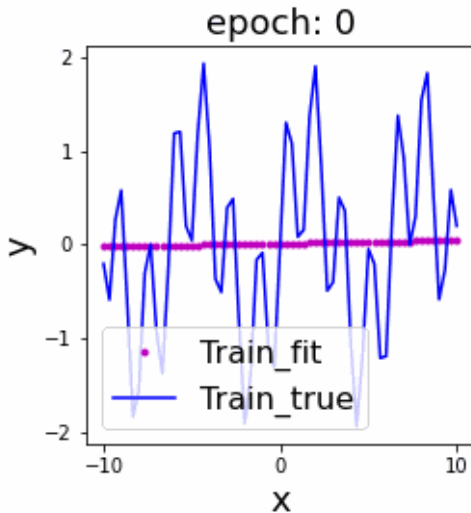
2Hz wavefields (x,z,sx), model: {4,4}, 96 param.

$$u = f(x_s, x, z; \theta)$$



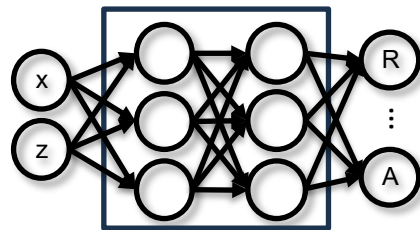
# Low frequency bias

Jean-Baptiste Joseph  
Fourier (1768-1830)



# For images

Fitting

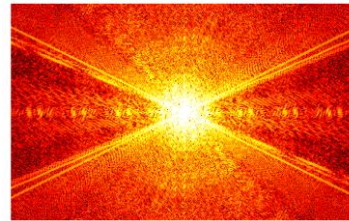
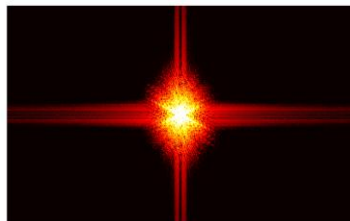


Data  
domain



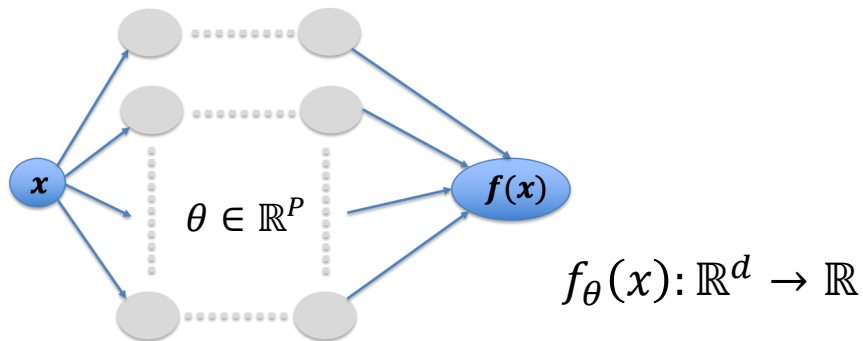
Epoch

Wavenumber  
Domain





# Insights: neural tangent kernel (NTK)

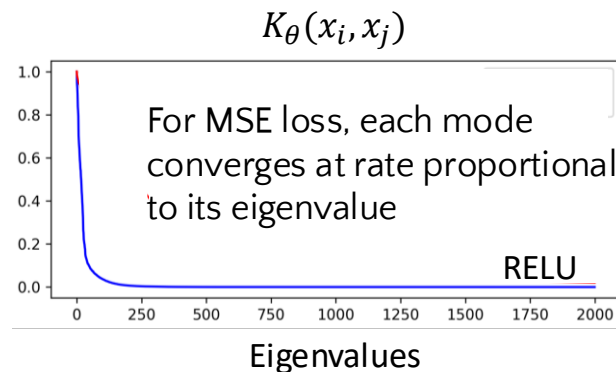
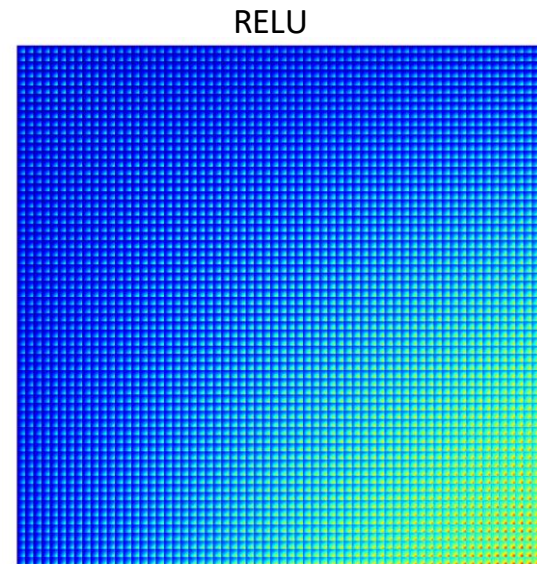


$$K_\theta(x, x') = \nabla_\theta f_\theta(x)^T \nabla_\theta f_\theta(x')$$

$$K_\theta \in \mathbb{R}^{N \times N}, \quad (K_\theta)_{ij} = K_\theta(x_i, x_j)$$

- **Infinite neural layer width assumption**

Jacot, A., Gabriel, F., & Hongler, C. (2018). Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)* (pp. 8571–8580). Curran Associates, Inc.



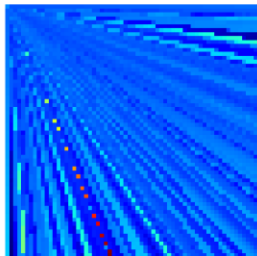


# Eigenvectors (ReLU)

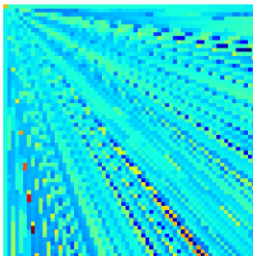
ReLU 1th



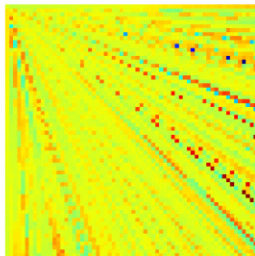
ReLU 51th



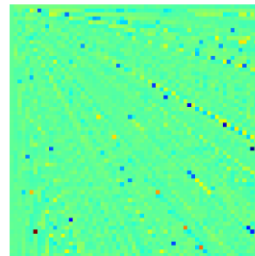
ReLU 101th



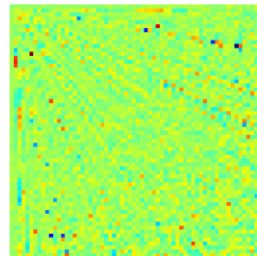
ReLU 151th



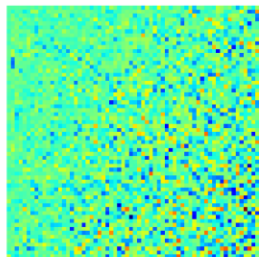
ReLU 201th



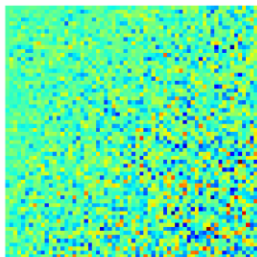
ReLU 251th



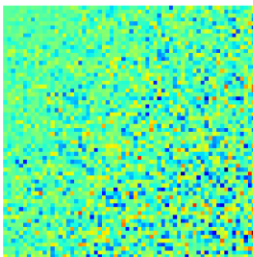
ReLU 301th



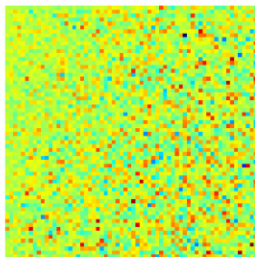
ReLU 351th



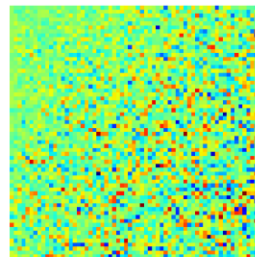
ReLU 401th



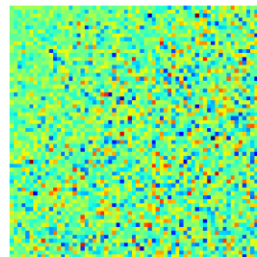
ReLU 451th



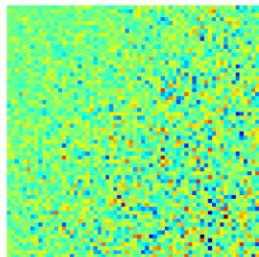
ReLU 501th



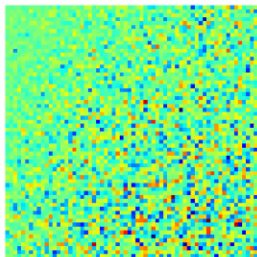
ReLU 551th



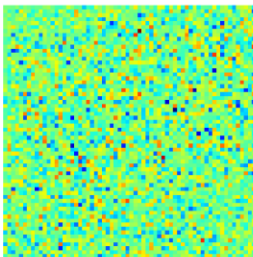
ReLU 601th



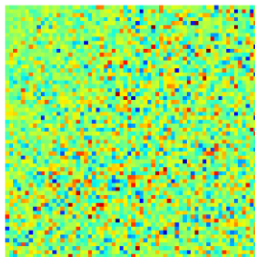
ReLU 651th



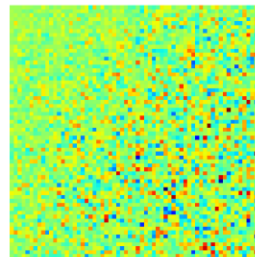
ReLU 701th



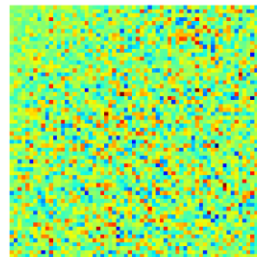
ReLU 751th



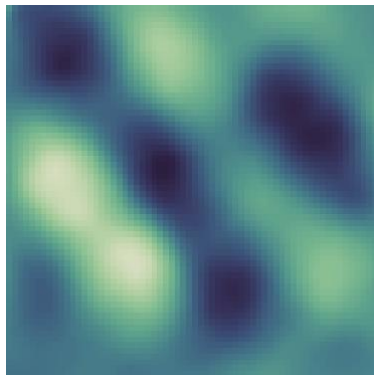
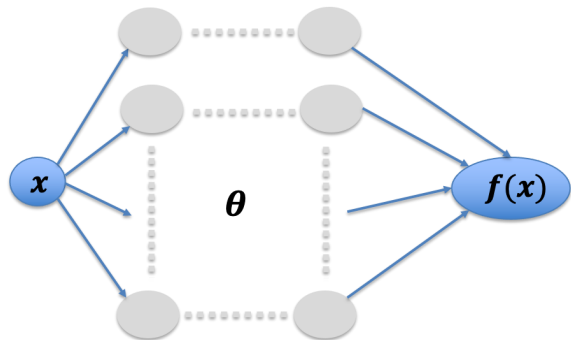
ReLU 801th



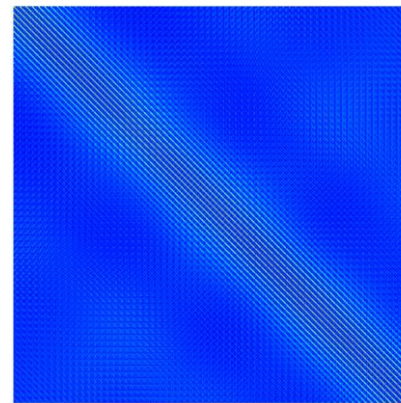
ReLU 851th



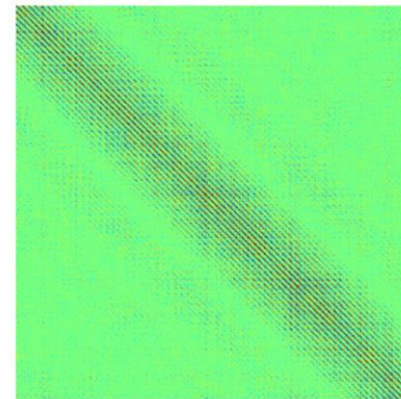
# Initializations



SIRON

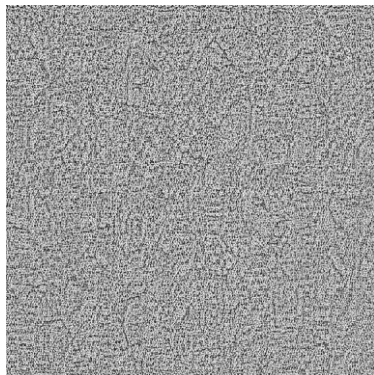
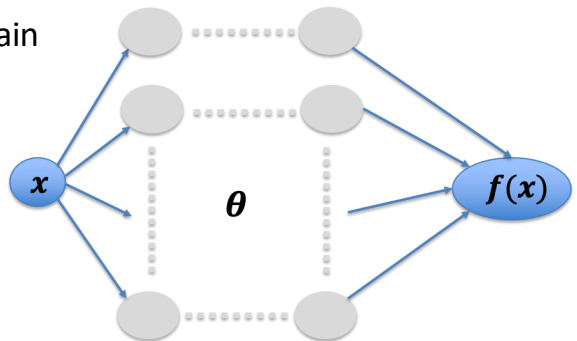


Pretrained SIRON



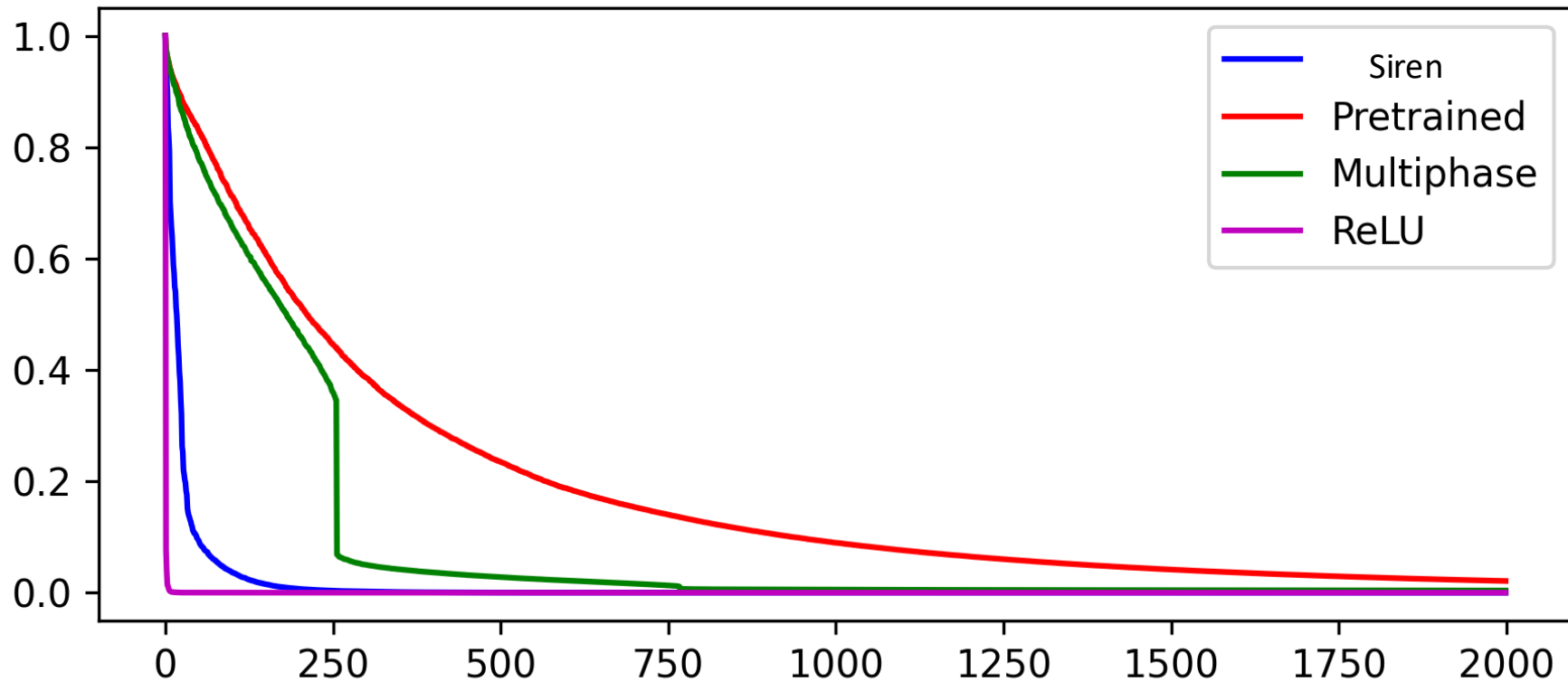
$K_{\theta}(x_i, x_j)$

Pretrain



# First 2000 Eigenvalues

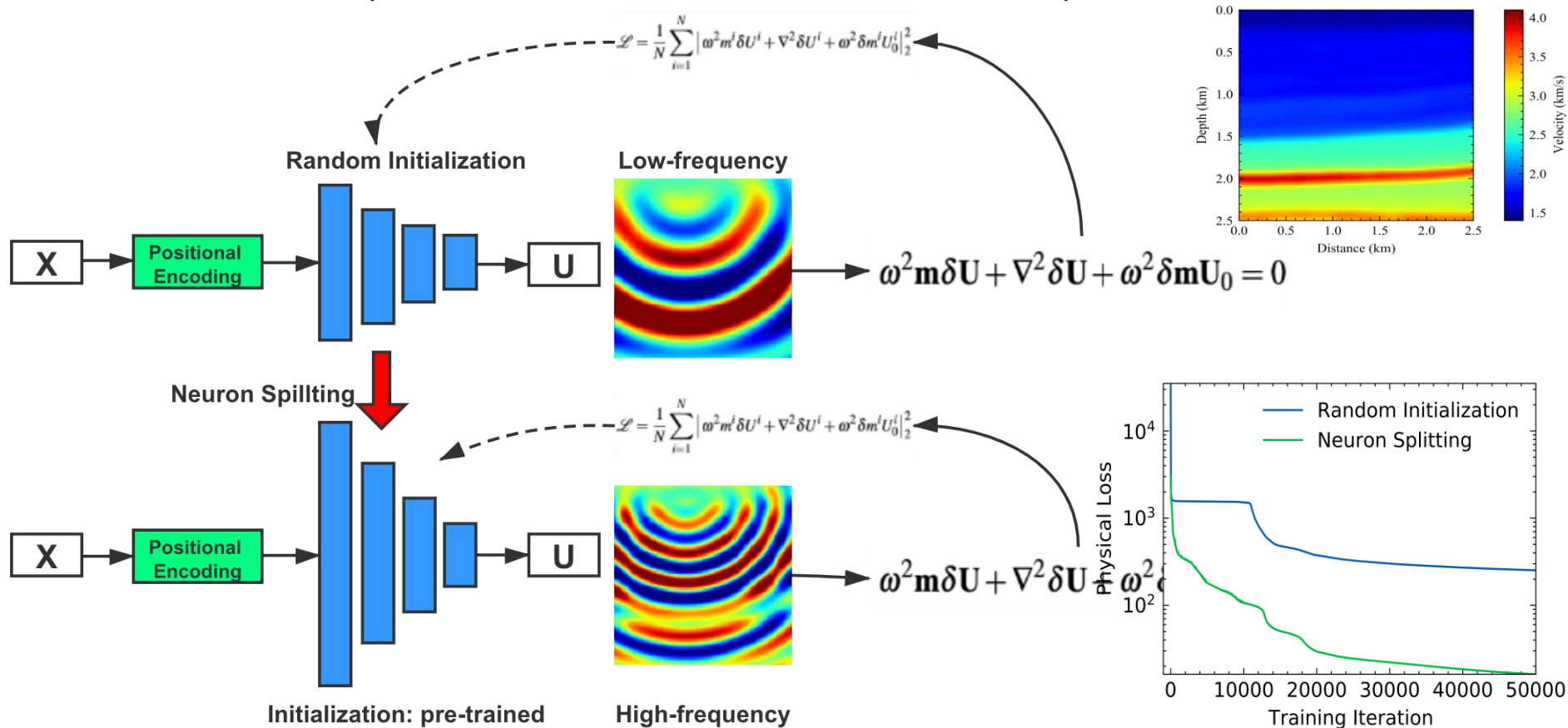
Eigenvalues



# PINNup

- **Low frequency bias** (N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. A. Hamprecht, Y. Bengio, and A. Courville, “On the spectral bias of neural networks,” arXiv, no. 1, 2018)
- **Low frequency to high**, in the spirit of FWI
- **Neuron Splitting** (Q. Liu, L. Wu, and D. Wang, “Splitting steepest descent for growing neural architectures,” arXiv preprint arXiv:1910.02366, 2019)

# Frequency upscaling and Neuron splitting



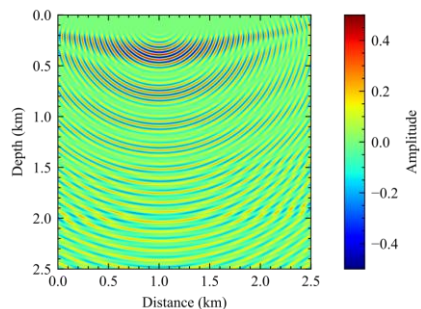
Huang and Alkhalifah, 2021, PINNup: Robust neural network wavefield solutions using frequency upscaling and neuron splitting, arXiv.



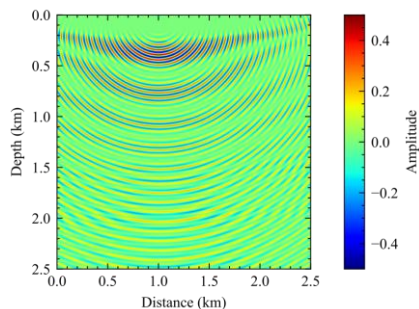
# 3D wavefields ( $x, z, x_s, 32 \text{ Hz}$ ) $\{1024, 1024\}$

Real parts

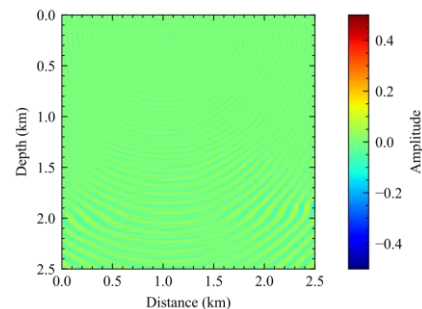
Numerical solution



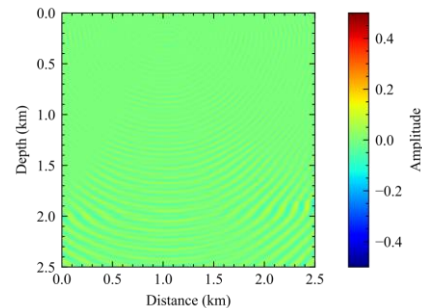
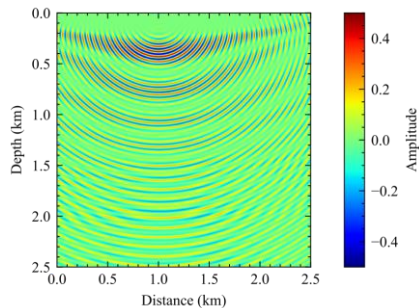
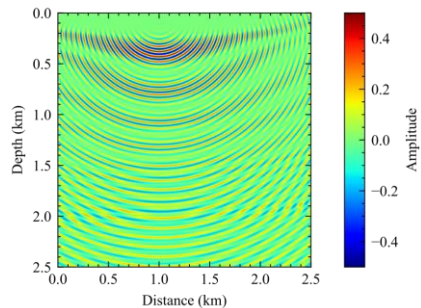
PINNup



Residuals

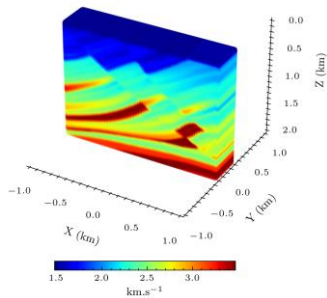


Imaginary parts

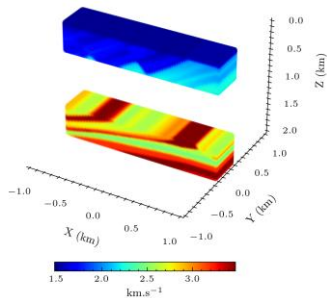


# PINN's interpolation properties

Sliced Original\* Velocity



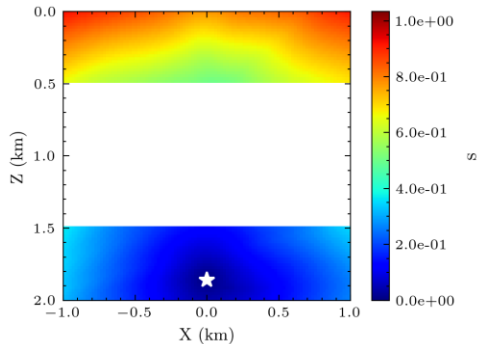
Input to PINNs



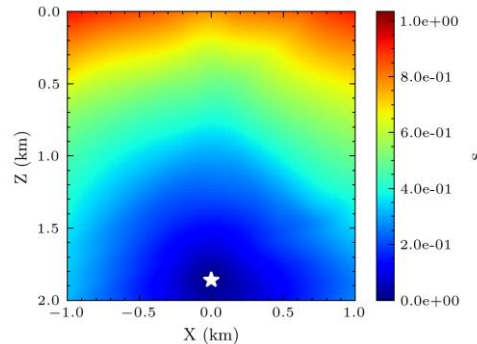
50% missing velocity

Input (with gaps)

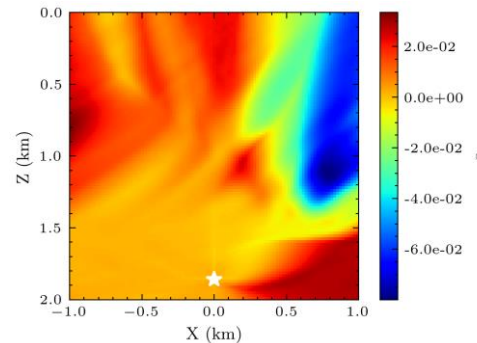
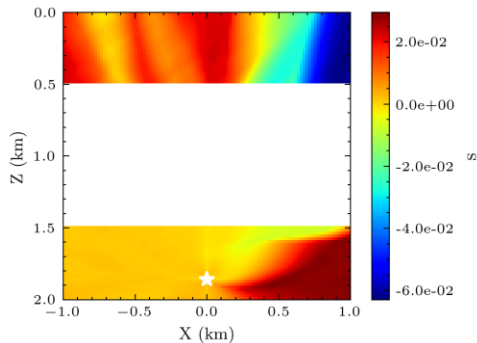
PINNs Calculation



Original Domain



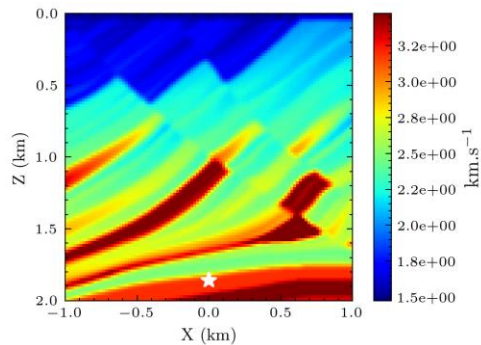
Traveltime error



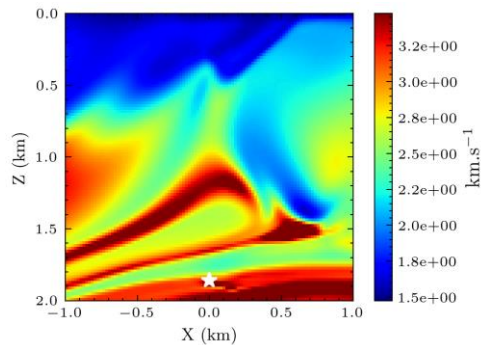


# The velocity model

Original Velocity

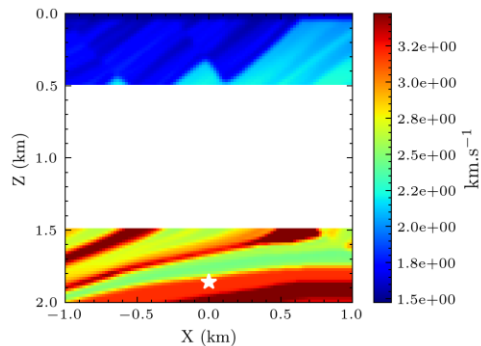


PINNs Output

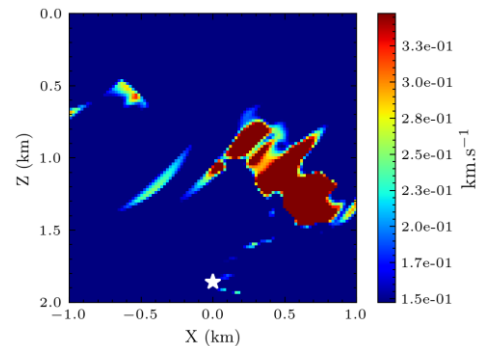


**50% missing velocity**

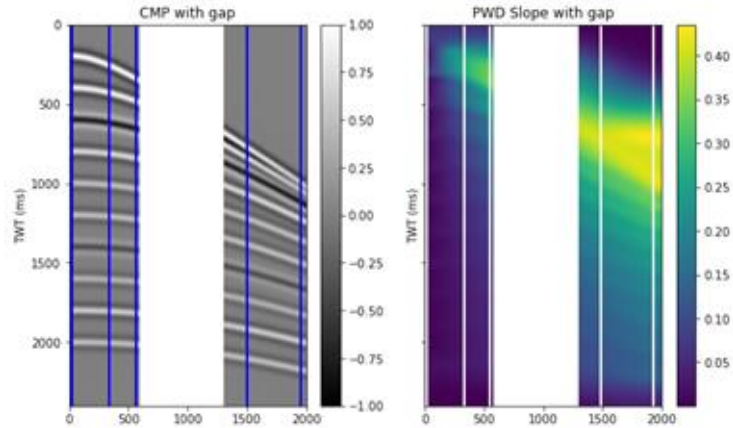
Input Velocity



Relative Error

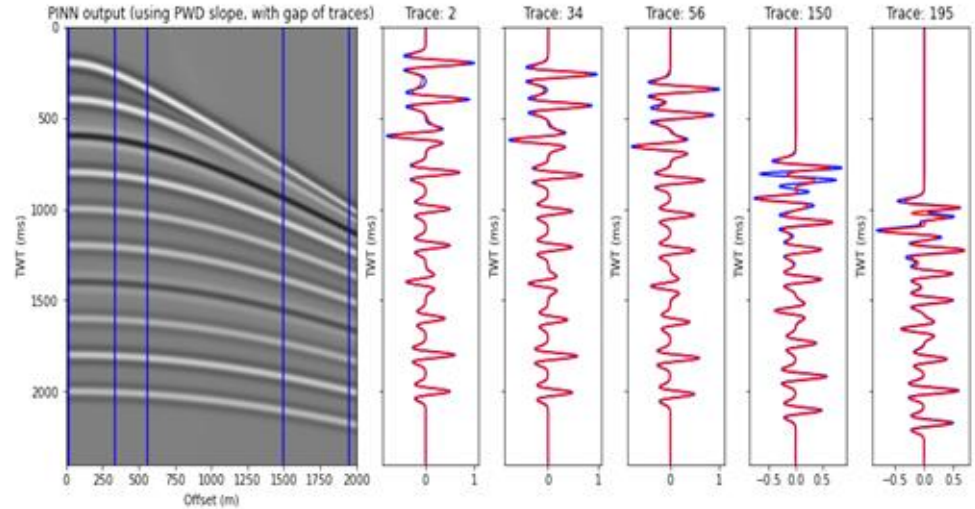


# PINNs for geophysical applications – Seismic interpolation

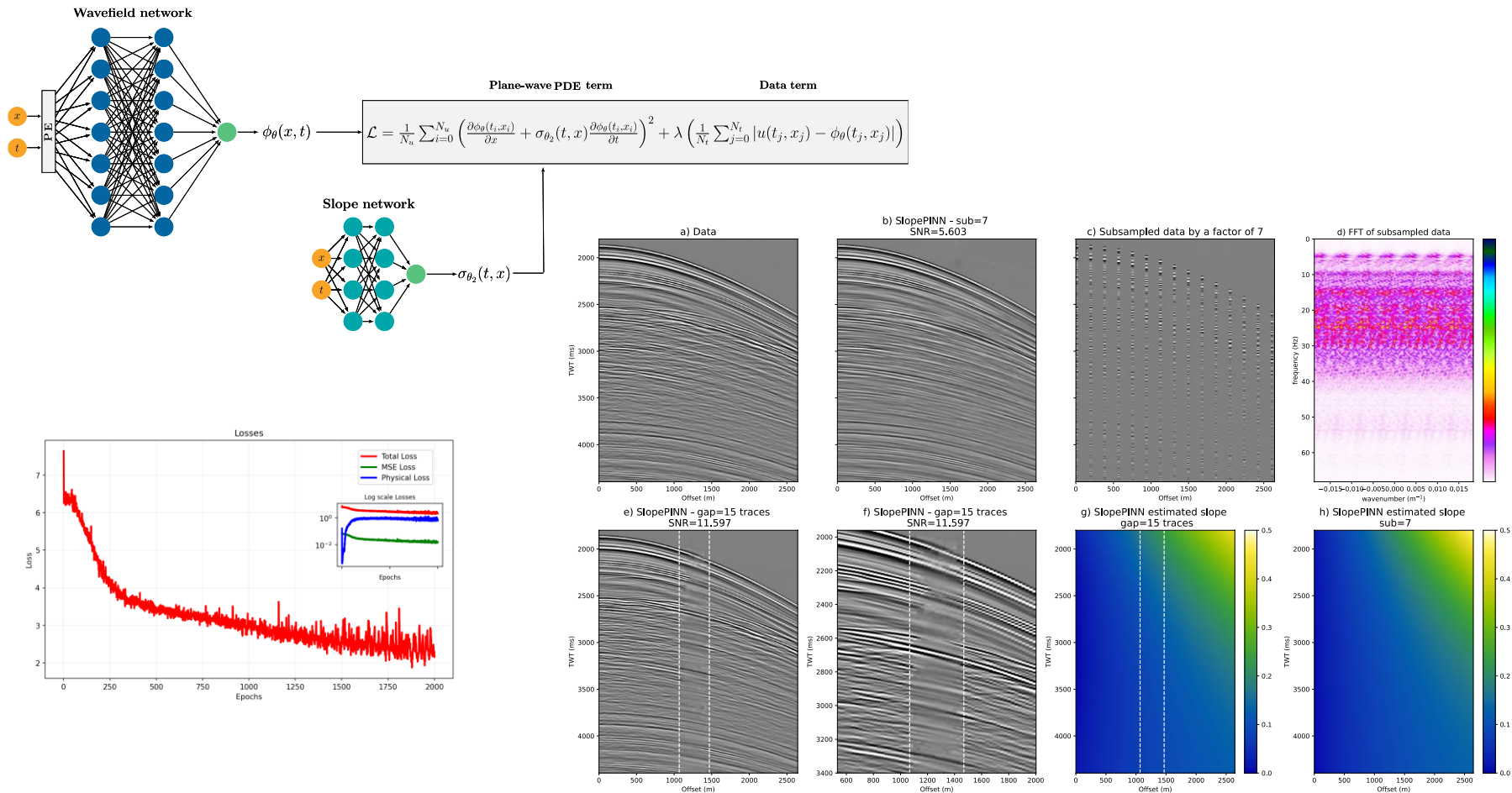


Network:  $u = f(x, t; \theta)$

$$PDE_u = u_x + \sigma u_t$$



# SlopePINN: physics-informed neural network slope estimation and interpolation with positional encoding



# Imaging

The governing equation:

$$\frac{\omega^2}{v^2(x,z)} u(x, z, \omega) + \nabla^2 u(x, z, \omega) = 0$$

subject to the boundary condition:

$$u(x, z, \omega) = D(x, \omega)$$

The time=0 imaging condition:

$$I(x, z) = \int u(x, z, \omega) d\omega$$

# PINN imaging

The conventional PINN loss:

$$L = \frac{1}{N} \sum_{i=1}^N \left| \frac{\omega^2}{v^2} u^i + \nabla^2 u^i \right| + a \frac{1}{N_b} \sum_{i=1}^{N_b} |D^i - u^i|$$

Over random  $(x_i, z_i, \omega_i)$  for  $i = 1, N$ ,  
and over recorded  $(x_i, \omega_i)$  for  $i = 1, N_b$

# Hard constraints

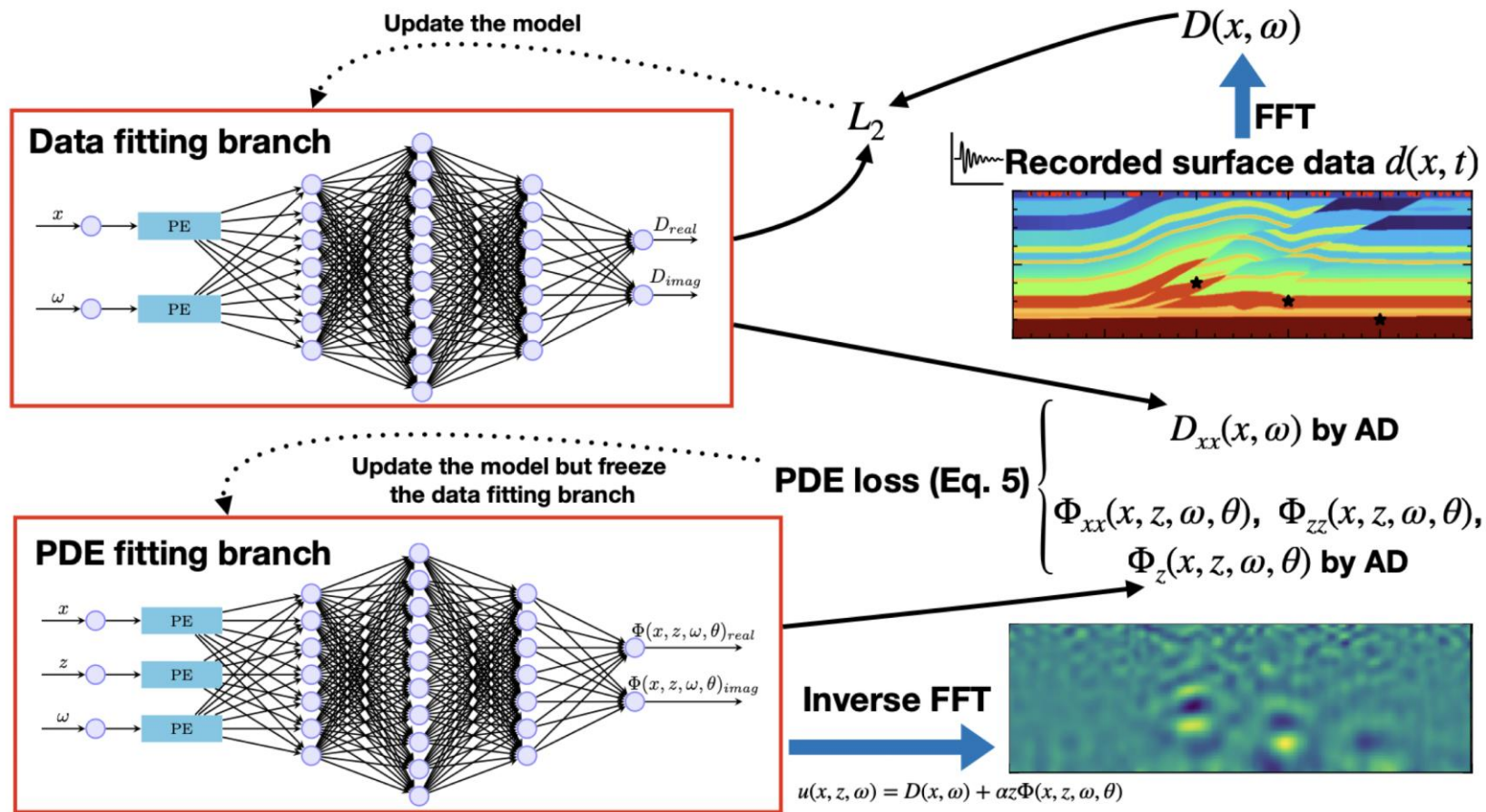
To inherently include the boundary condition:

$$u(x, z, \omega) = D(x, \omega) + z\Phi(x, z, \omega, \theta),$$

The corresponding Loss:

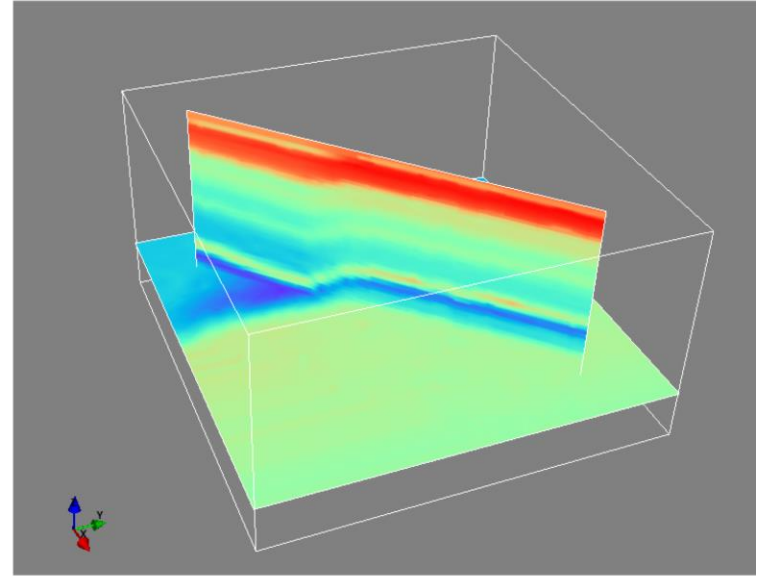
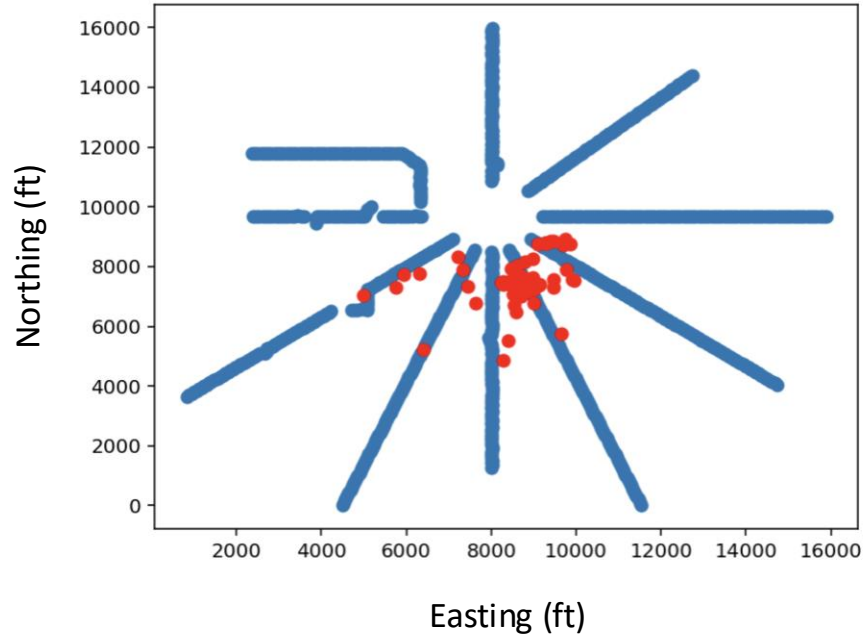
$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left| \mathbf{z}^i \frac{(\omega^i)^2}{(v^i)^2} \Phi(\mathbf{x}^i, \mathbf{z}^i, \omega^i, \theta) + \mathbf{z}^i \nabla^2 \Phi(\mathbf{x}^i, \mathbf{z}^i, \omega^i, \theta) + \right. \\ \left. 2\nabla_z \Phi(\mathbf{x}^i, \mathbf{z}^i, \omega^i, \theta) + \frac{(\omega^i)^2}{(v^i)^2} D(\mathbf{x}^i, \omega^i) + 2\nabla_x^2 D(\mathbf{x}^i, \omega^i) \right|_2^2$$

# Application on source imaging

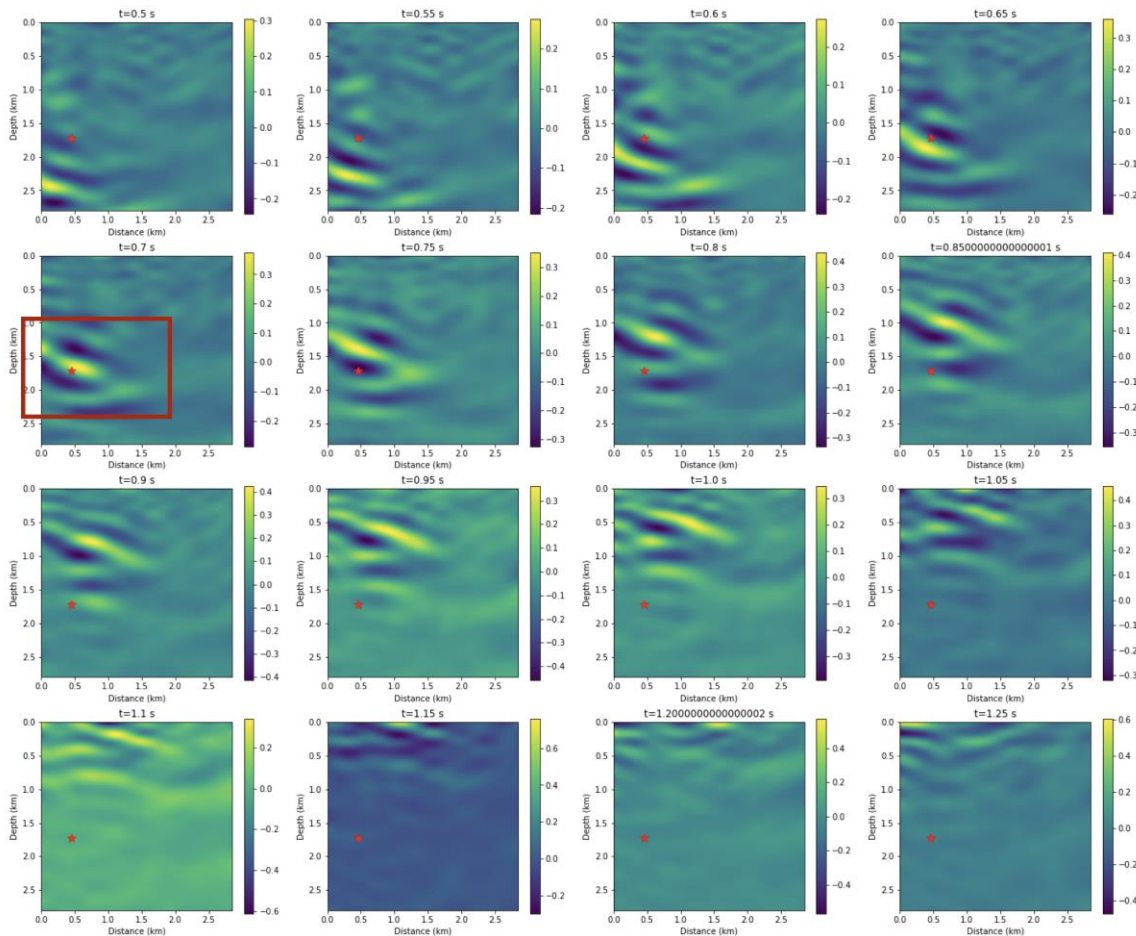




# Field tests: (Oklahoma Arkoma Basin)



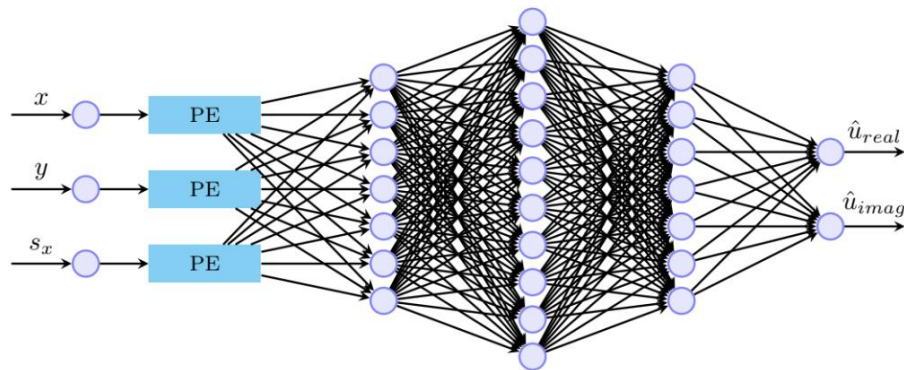
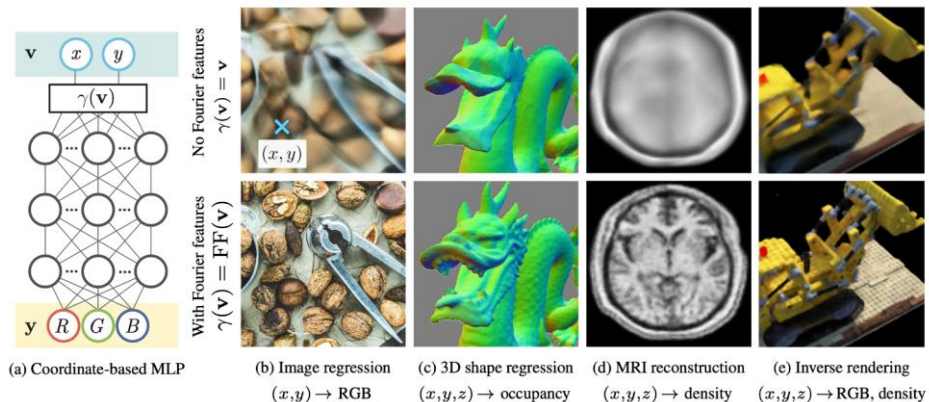
# Snapshots in the time domain



*Cost? The elephant in the room.*

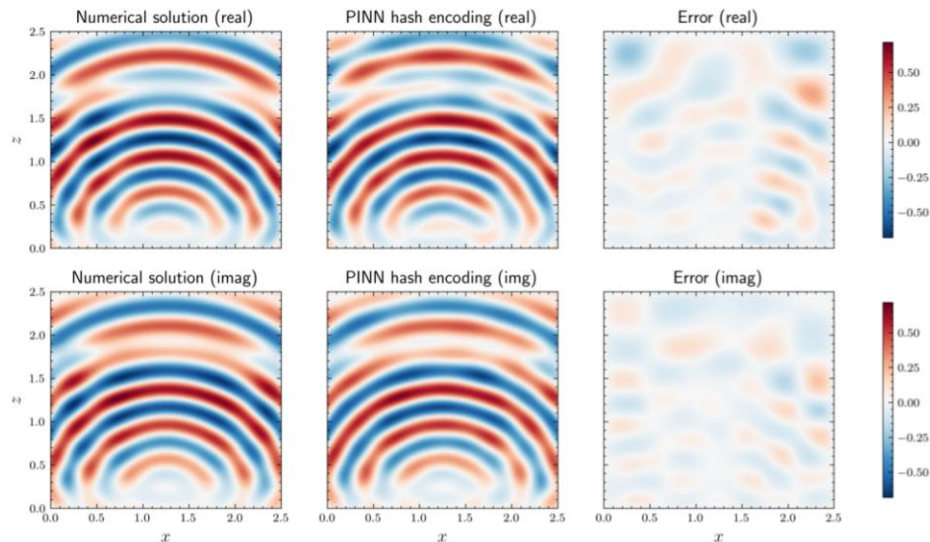
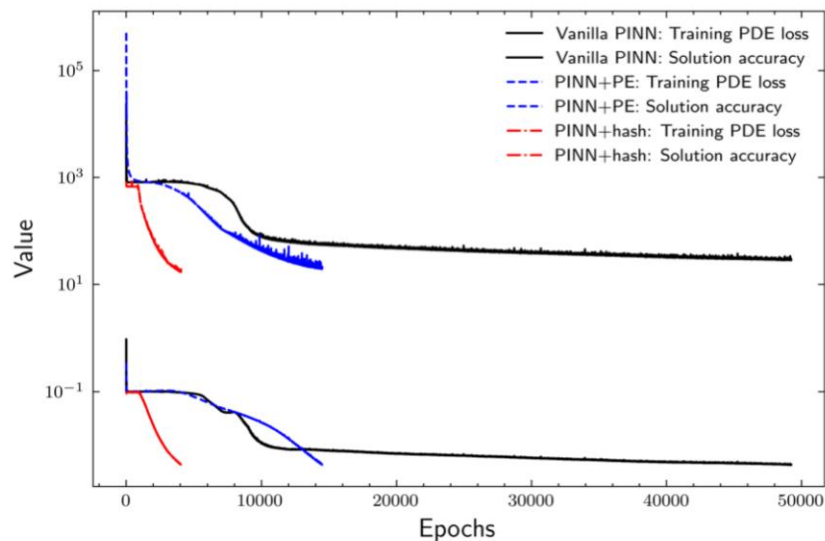
# Encoding

- High dimensional representation of the scalar input.
- More weight to the location within the network (point source versus plane source).
- Positional encoding, borrowed from Transformers (Fourier encoding in NeRF).



# Helmholtz equation (scattered wavefield)

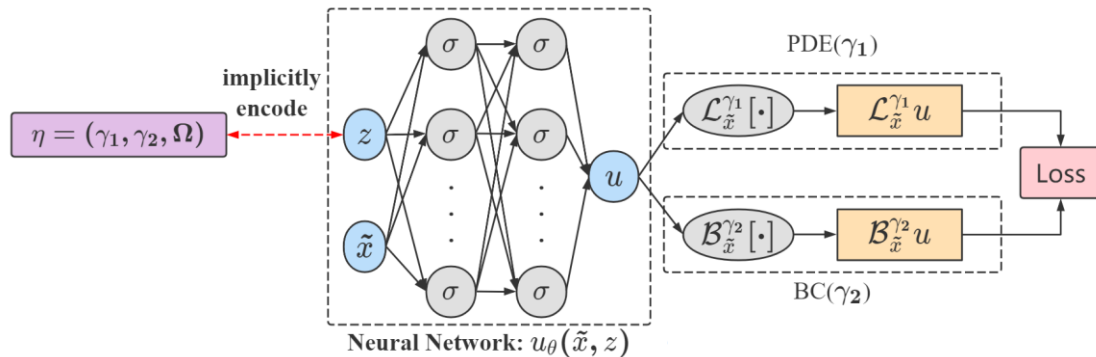
Reduced from 491 s to 39 s



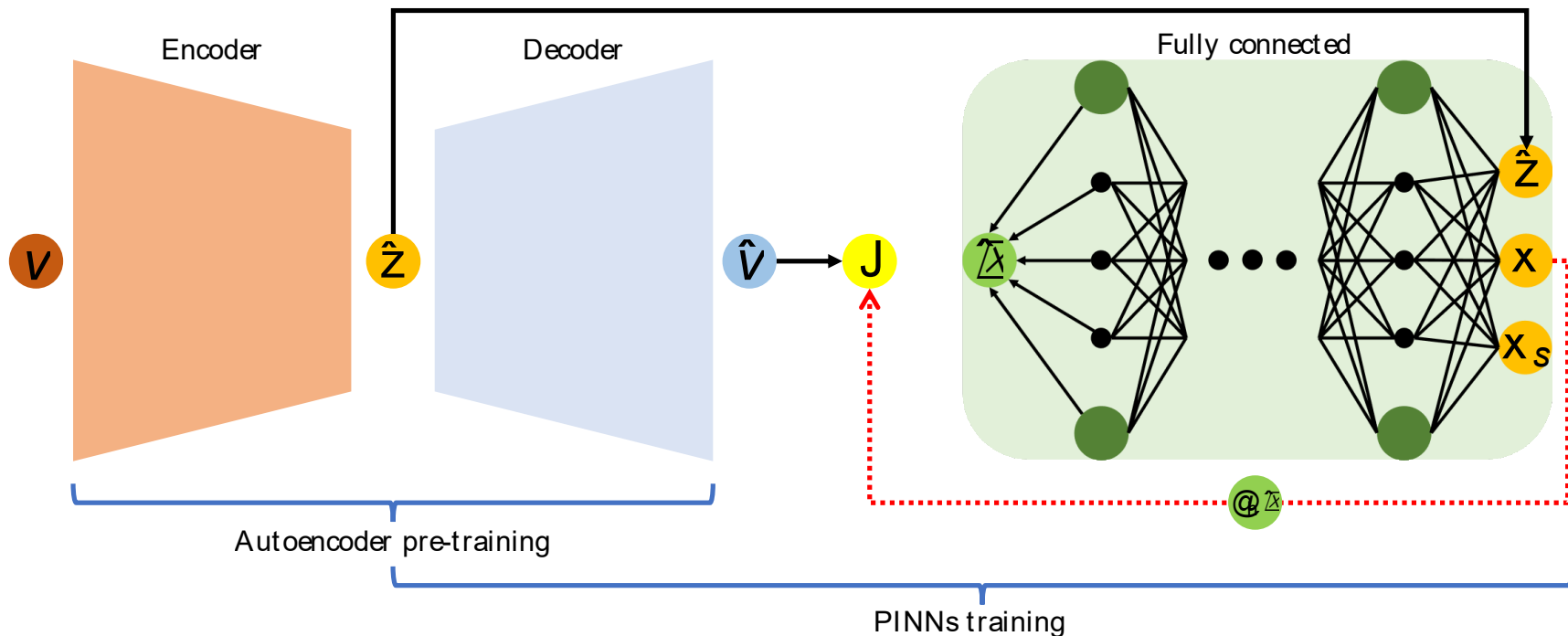
$T=8$ , MLP: {144,144,144}, MLP with hash encoding: {128,128,128}

# Generalization

- -Solutions for a specific PDE parameter
- -Transfer learning?
- -Meta-PDE  $\rightarrow$  learning a good initialization
- -Meta-Auto-Decoder



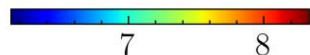
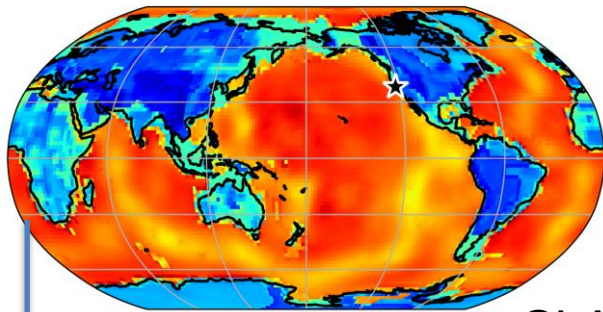
# LatentPINN: a generator PINN





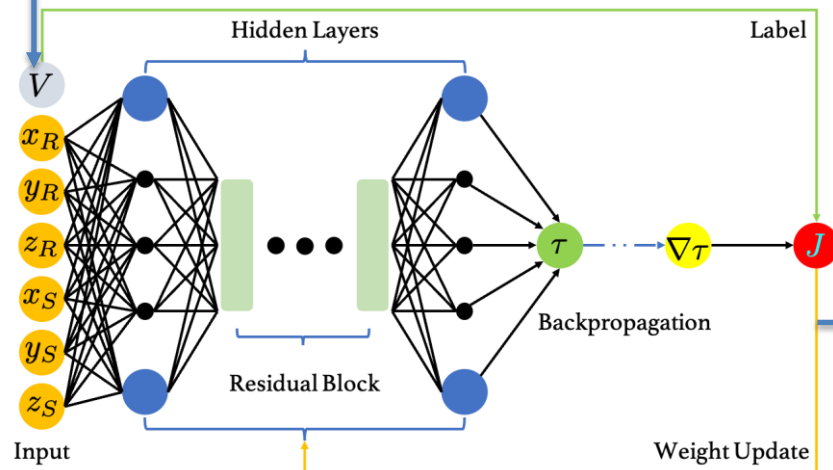
# Embedding the Earth velocity in an NN

depth = 24.0

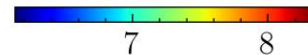
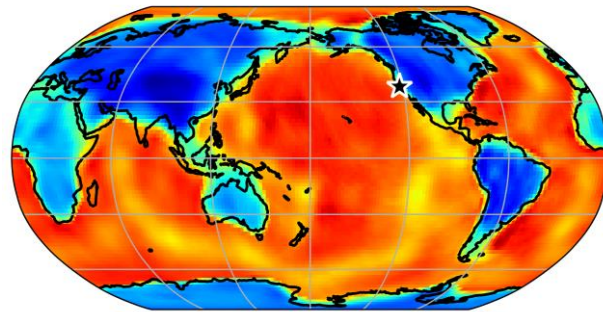


Vertically Polarized P-wave  
Velocity [km.s<sup>-1</sup>]

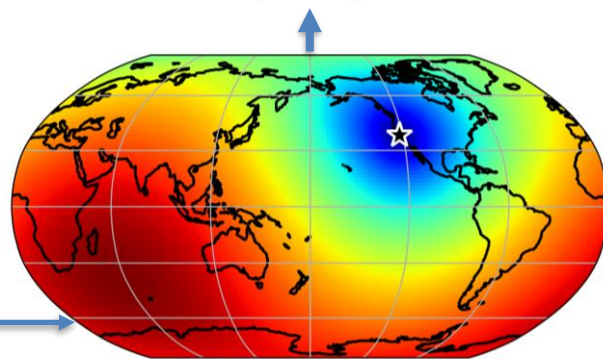
GLAD-M25 model



depth = 24.0



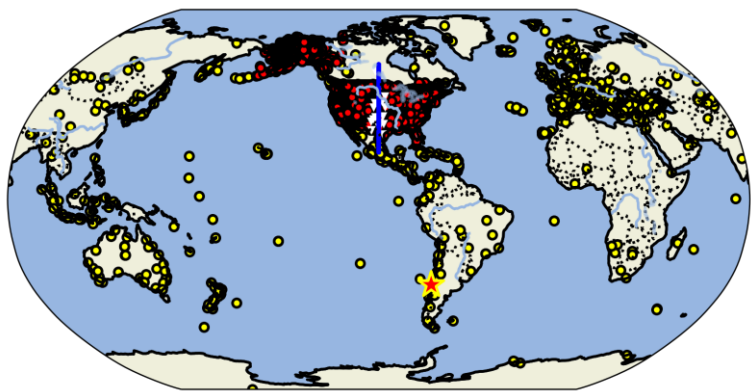
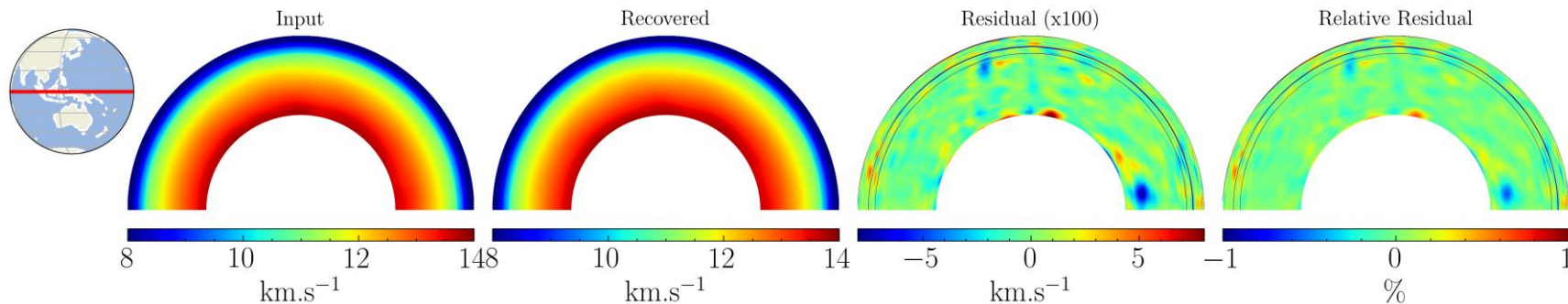
P-wave Velocity (Recovered)  
[km.s<sup>-1</sup>]



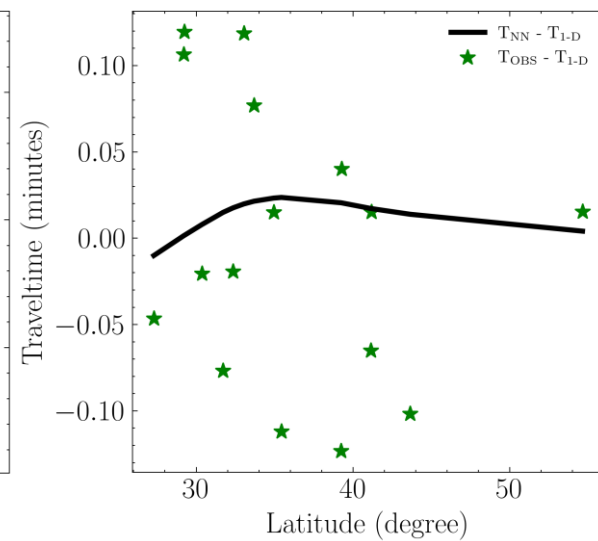
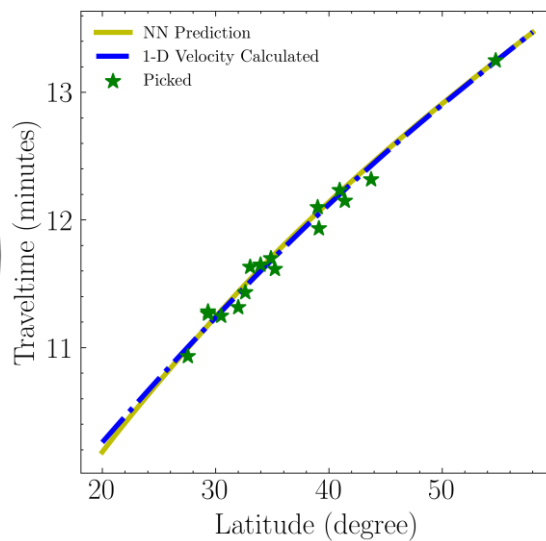
First-Arrival Travel Time



# Accuracy and interpolation power



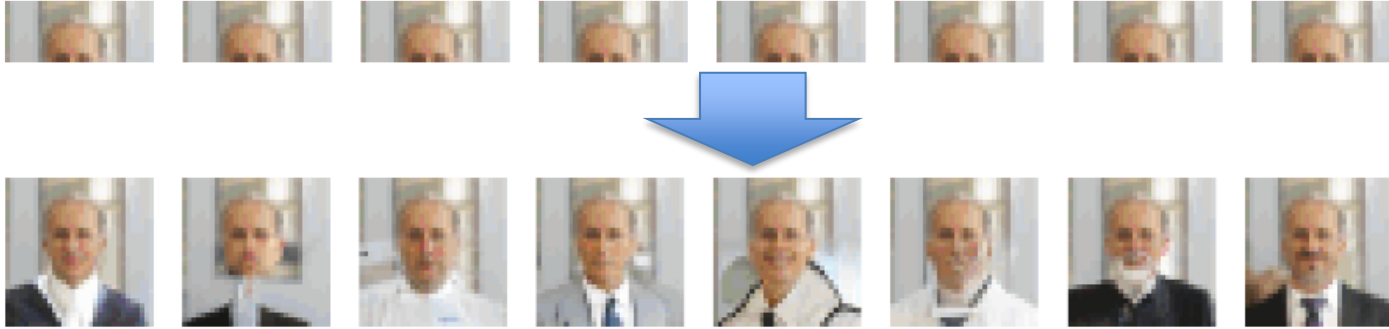
Earthquake in Chile



# Key points

- *PINNs are nice, but challenges exist*
- *Positional encoding, PINNup, and many other*
- *Interpolation properties*
- *Hash encoding  $\rightarrow$  fast convergence*
- *latentPINN  $\rightarrow$  generalization*

# Acknowledgements



- My group who inspire me all the time
- We thank KAUST for its continuous support of our work
- Thank you for being here

A wide-angle photograph of a desert landscape. In the foreground, there are prominent, layered rock formations in shades of tan and brown, showing clear horizontal sedimentary patterns. The rock face is rugged and weathered. Beyond the rocks, a vast, flat valley stretches out, with some sparse vegetation and a few small structures visible in the distance. The horizon is marked by a range of low, hazy mountains under a clear sky. The overall scene is bright and sunny, suggesting a clear day.

***Thank You***

30 17:50